

CRUD

Python - Django - Pydantic



18/05/2025 - IA - Gemini

Creación de CRUD en Python con Django y Pydantic

Prompts

COMO PROGRAMADOR EXPERTO, CREA UN CRUD Y LOS ARCHIVOS NECESARIOS PARA QUE UTILICE LA ESTRUCTURA DE DJANGO MODEL TEMPLATE VIEW (MTV), CREA LOS MODELS, TEMPLATES, VIEW, URLS, FORMS Y APP CORRESPONDIENTES, USARAS COMO BASE DE DATOS POSTGRES. REALIZA EL CONTROL DE LAS VALIDACIONES CON PYDANTIC VERSION 2.11.5 A TRAVES DE MENSAJES DE ERROR PERSONALIZADOS USANDO DJANGO, LOS MENSAJES DE ERROR SE DEBEN MOSTRAR USANDO LA LIBRERÍA SWEETALERT2

//

CON PYDANTIC NO USARAS "@validator" PORQUE ESTA OBSOLETA, DEBES USAR "field_validator" y "model_validator"

//

EL NOMBRE DE LA CARPETA QUE CONTENDRA MI PROYECTO SERA "mis_empleados"

//

MANTEN ESTA ESTRUCTURA DE ARCHIVO, EL NOMBRE DEL ENTORNO VIRTUAL SERA "empleados_dj521" Y SE CREARA DENTRO DE LA CARPETA "Desarrollo Py\Python-EntornosV"

//

EL NOMBRE DE LA BASE DE DATOS SERA "db_empleados_521"

USARAS VARIABLES DE ENTORNO PARA LAS CREDENCIALES DE LA BASE DE DATOS

INDICAME COMO CREAR LA BASE DE DATOS DESDE EL TERMINAL

//

EL NOMBRE DEL PROYECTO SERA "form_empleados"

//

CREA EL PROYECTO SIN QUE SE GENERE UNA SUBCARPETA

//

MANTEN ESTA ESTRUCTRA DE ARCHVO, EL NOMBRE DE MIS APP SERAN "empleados" y "usuarios" Y SE CREARAN DENTRO DE LA CARPETA "mis_empleados",

//

ESTOS SON LOS DATOS QUE SE PODRAN CREAR, LEER, ACTUALIZAR Y ELIMINAR

//

NOMBRES

APELLIDOS

EDAD

FECHA

EMAIL

TELEFONO_CASA

TELEFONO_MOVIL

SUELDO BASE

COMISION

SUELDO BRUTO

PASSWORD

FOTO

//

CADA INPUT TENDRA LAS SIGUIENTES VALIDACIONES

//

NOMBRES, DEBE TENER UN MAXIMO DE 60 CARACTERES.

APELLIDOS, DEBE TENER UN MAXIMO DE 60 CARACTERES.

EDAD, NO DEBE SER MAYOR DE 100 AÑOS.

FECHA, SE DEBE INGRESAR EN FORMATO DD-MM-AAA.

EMAIL, DEBE TENER FORMATO CORRECTO DE CORREO ELECTRONICO.

TELEFONO_CASA, DEBERA USAR UN FORMATO DE TELEFONO FIJO DE ESPAÑA.

TELEFONO_MOVIL, DEBERA USAR UN FORMATO DE TELEFONO MOVIL DE ESPAÑA.

SUELDO BASE, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

COMISION, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

SUELDO BRUTO, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

SUELDO BRUTO, SE DEBE CALCULAR Y ACTUALIZAR AUTOMATICAMENTE CON LA SUMA DE SUELDO BASE + COMISION.

PASSWORD, DEBE TENER COMO MINIMO 8 CARACTERES.

FOTO, SIEMPRE SE DEBE CARGAR UNA IMAGEN,

//

FOTO, COLOCAR UN BOTON QUE PERMITA SELECCIONAR Y CARGAR UNA IMAGEN

//

TODOS LOS INPUT SON DE INGRESO DE DATOS OBLIGATORIO

//

COLOCA UN ICONS EN EL CAMPO PASSWORD PARA VISUALIZAR U OCULAR LO INGRESADO

//

IMPLEMENTA EL HASHING DE CONTRASEÑAS. USA LAS FUNCIONES DE HASHING QUE DJANGO PROPORCIONA (DJANGO.CONTRIB.AUTH.HASHERS)

//

AÑADE AUTENTICACIÓN DE USUARIOS. CREA MODELOS DE USUARIO PERSONALIZADOS (USA EL USER DE DJANGO), FORMULARIOS DE REGISTRO E INICIO DE SESIÓN, Y VISTAS PARA MANEJAR LA AUTENTICACIÓN.

//

AÑADE UN PANEL DE ADMINISTRACIÓN PERSONALIZADO PARA GESTIONAR LOS EMPLEADOS.

//

PARA QUE EL PROYECTO SEA RESPONSIVE UTILIZA BOOTSTRAP V5.0

//

UTILIZA LAS LIBRERIAS BOOTSTRAP V5.0, SWEETALERT2 Y FONT AWESOME FREE DESDE LA NUBE.

//

AGREGA ICONS DE LA LIBRERÍA **FONT AWESOME FREE** A LOS BOTONES DE CREAR, EDITAR Y ELIMINAR ASI COMO A LOS MENSAJES DE ERROR

//

PARA INSTALAR LAS DEPENDENCIAS CREA UN ARCHIVO, Y PONLE DE NOMBRE "requirements.txt".

AGREGA ESTE PAQUETE "python-dotenv" Y HAZ SU IMPLEMENTACIÓN EN EL "settings.py"

//

MUESTRAME EL CONTENIDO DEL ARCHIVO .env

//

MEJORA LA INTERFAZ DE USUARIO: AUNQUE YA ESTAS USANDO BOOTSTRAP 5, EXPLORA FORMAS DE MEJORAR AÚN MÁS LA INTERFAZ DE USUARIO, COMO PAGINACIÓN PARA LISTAS LARGAS, ESTILOS PERSONALIZADOS ADICIONALES, O EL USO DE JAVASCRIPT PARA INTERACCIONES MÁS DINÁMICAS (APARTE DEL TOGGLE DE LA CONTRASEÑA Y SWEETALERT), RECUERDA TODAS LAS FUNCIONES JS SE DEBEN GUARDAR EN UN ARCHIVO EXTERNO DENTRO DE LA CARPETA "STATIC"

//

IMPLEMENTA LA ELIMINACIÓN SUAVE (SOFT DELETE): EN LUGAR DE ELIMINAR PERMANENTEMENTE LOS REGISTROS, IMPLEMENTA UNA ELIMINACIÓN SUAVE, DONDE SE MARCA UN REGISTRO COMO ELIMINADO PERO PERMANECE EN LA BASE DE DATOS.

//

ESPECIFICA EL NOMBRE DE CADA ARCHIVO QUE SE VAYA A UTILIZAR PARA CREAR LO SOLICITADO.

//

EL CONTENIDO DE LOS CODIGOS CSS Y JS DEBEN ESTAR SEPARADOS POR ARCHIVOS Y CARPETAS.

//

VERIFICA QUE TODOS LOS CODIGOS ESTEN ACTUALIZADOS, NO USES NADA OBSOLETO

//

MUESTRAME LA ESTRUCTURA DE ARCHIVOS.

//

AL FINALIZAR COLOCA EL TITULO "DESARROLLANDO CRUD CON PYTHON - DJANGO - PYDANTIC – EXPLICACION DETALLADA" Y MUÉSTRAME EN DETALLE Y PASO A PASO TODOS LOS ARCHIVOS COMPLETOS Y SU RESPECTIVA EXPLICACIÓN PARA INSTALAR FRAMEWORK, LIBRERÍAS Y ENTORNOS, ASÍ COMO LA EXPLICACIÓN DE CADA ARCHIVO. MUESTRALO EN ESTE ORDEN :

1 - Estructura de Archivos del Proyecto

2 - Instalación y Configuración

- Crear un Entorno Virtual
- Muéstrame el contenido del archivo requirements.txt.
- Instalar Django, Psycpg2 (para PostgreSQL), Pydantic v2 y todos los paquetes y dependencias que sean necesarios para este proyecto.
- Configurar la Base de Datos PostgreSQL
- Crear el Proyecto y las Aplicaciones
- Añadir las Aplicaciones a settings.py
- Crear y Aplicar las Migraciones
- Crear un Superusuario (para el panel de administración)
- Ejecutar el Servidor de Desarrollo

3 - Contenido de los Archivos

4 - Documentación del Proyecto

GENERA LA DOCUMENTACIÓN PARA EL PROYECTO, EXPLICANDO SU ESTRUCTURA, LOS MODELOS, LAS VISTAS, LOS FORMULARIOS, ETC.

//

Resultado



Como programador experto, te guiaré a través de la creación de un CRUD completo en Django utilizando la arquitectura MTV, con PostgreSQL como base de datos y validaciones con Pydantic y SweetAlert2.

(Este documento fue adaptado en base a la respuesta de la IA, identifico con "IA", lo generado automáticamente)

IA

Aquí te muestro la estructura de archivos que mantendremos para tu proyecto `mis_empleados`:

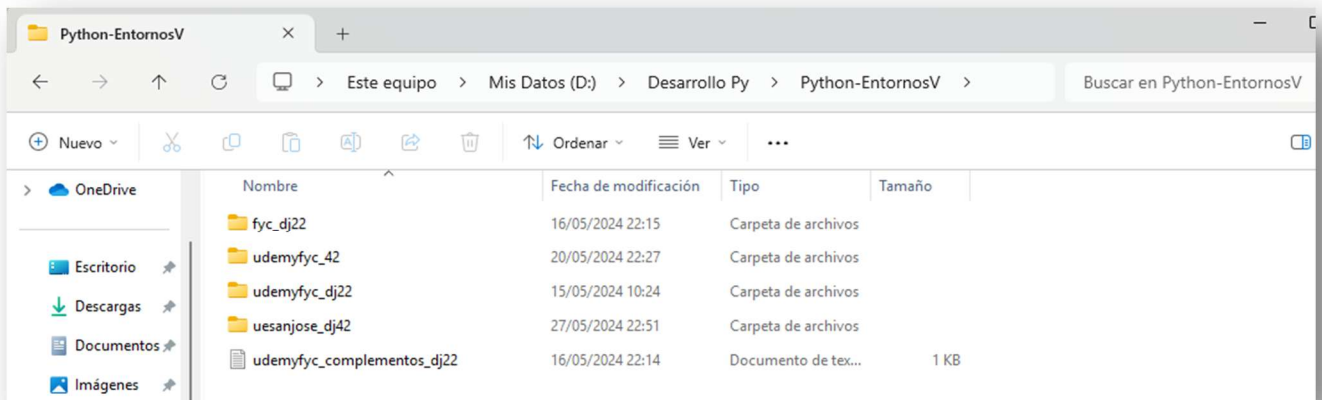
```
Desarrollo Py/  
└─ Python-EntornosV/  
    └─ empleados_dj521/ (Entorno virtual)
```

```
mis_empleados/  
├─ empleados/ # Aplicación 'empleados'  
│   ├── migrations/  
│   ├── templates/ # Contiene la plantilla base para el proyecto  
│   │   └─ base.html # RUTA CLAVE: mis_empleados/empleados/templates/base.html  
│   ├── __init__.py  
│   ├── admin.py  
│   ├── apps.py  
│   ├── forms.py  
│   ├── models.py  
│   ├── pydantic_models.py # Modificado para importaciones de field_validator y mo  
│   ├── tests.py  
│   ├── urls.py  
│   └─ views.py  
├─ form_empleados/ # Carpeta del proyecto principal de Django (contiene se  
│   ├── form_empleados/  
│   │   ├── __init__.py  
│   │   ├── asgi.py  
│   │   ├── settings.py # <--- MODIFICADO: DIRS en TEMPLATES, LOGIN_URL, STATIC  
│   │   ├── urls.py # Inclusión de 'usuarios.urls', 'empleados.urls', home_  
│   │   └─ wsgi.py  
│   ├── manage.py  
│   └─ db.sqlite3  
├─ static/ # <--- CARPETA STATIC GLOBAL (mis_empleados/static/)  
│   ├── css/  
│   │   └─ styles.css  
│   ├── img/ # Carpeta para imágenes  
│   └─ js/  
│       └─ scripts.js  
├─ usuarios/ # Aplicación 'usuarios'  
│   ├── migrations/  
│   ├── templates/ # Contiene plantillas de usuario directamente  
│   │   ├── login.html # RUTA CLAVE: mis_empleados/usuarios/templates/login.htm  
│   │   └─ register.html # RUTA CLAVE: mis_empleados/usuarios/templates/register  
│   ├── __init__.py  
│   ├── admin.py  
│   ├── apps.py  
│   ├── forms.py # Modificado para importaciones de field_validator y mo  
│   ├── models.py  
│   ├── tests.py  
│   ├── urls.py # name='login' para CustomLoginView  
│   └─ views.py # template_name='login.html'  
└─ requirements.txt # Añadido email_validator
```

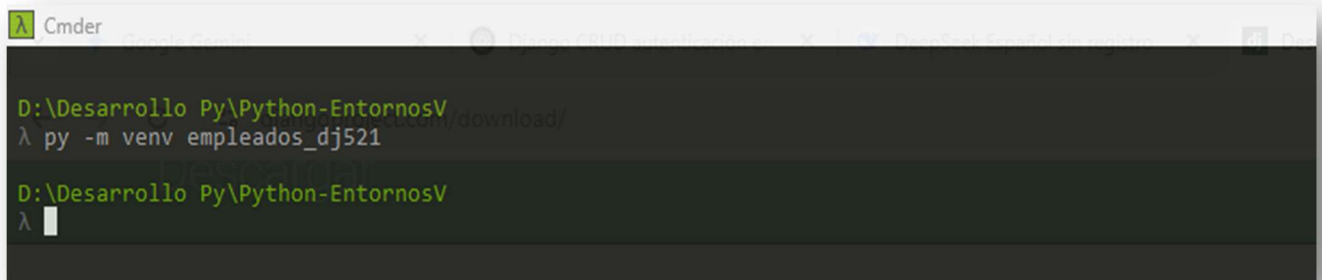
CONFIGURACION INICIAL

CREAR Y ACTIVAR ENTORNO VIRTUAL

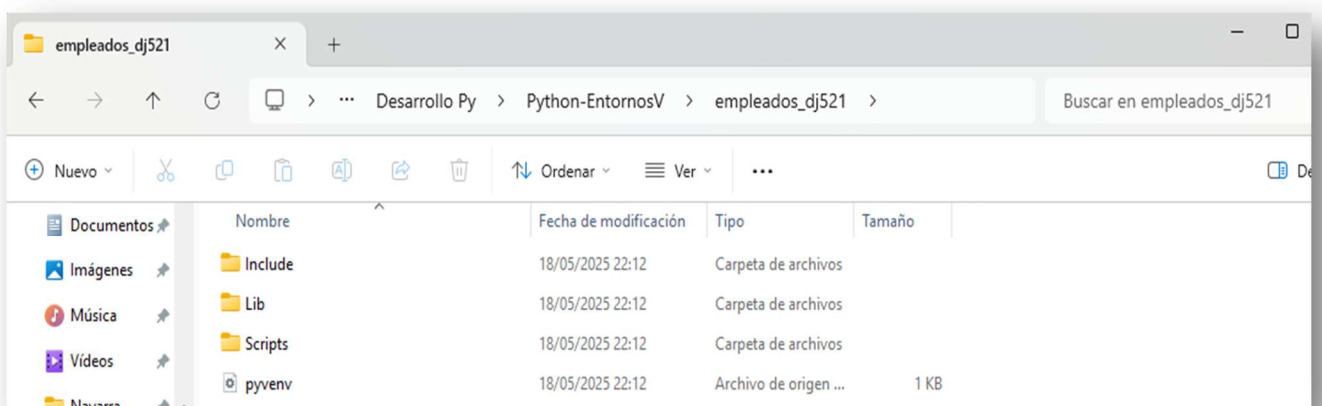
Para ello ingresamos a la carpeta donde tenemos todos nuestros [entornos virtuales](#) de Python, y abrimos nuestro terminal. (Puede usar como terminal, PowerShell, Terminal de Visual Studio Code o Cmder, recomiendo para este proyecto usar Cmder)



Creamos el entorno virtual : `py -m venv empleados_dj521` (`empleados_dj521` es el nombre del entorno)



Automáticamente se crea la carpeta de nuestro entorno virtual



Explicación:

Un [entorno virtual](#) en Python es un espacio aislado que permite gestionar las dependencias de un proyecto específico sin afectar a otros proyectos o a la instalación global de Python en tu sistema. Es como tener una instalación independiente de Python para cada proyecto, lo que facilita la gestión de diferentes versiones de librerías y evita conflictos.

En detalle:

- **Aislamiento:**
Cada entorno virtual tiene su propio directorio que contiene una copia independiente de Python y de las librerías que se instalen dentro de él.

- **Gestión de dependencias:**
Permite instalar y gestionar las librerías específicas que requiere un proyecto sin afectar a otras aplicaciones o a la instalación global de Python.
- **Evita conflictos:**
Al aislar las dependencias de cada proyecto, se evitan los conflictos que pueden surgir al tener diferentes versiones de la misma librería instaladas en el sistema.
- **Facilita el trabajo con múltiples proyectos:**
Permite trabajar en diferentes proyectos con diferentes versiones de librerías sin problemas de compatibilidad.

Herramientas para crear entornos virtuales:

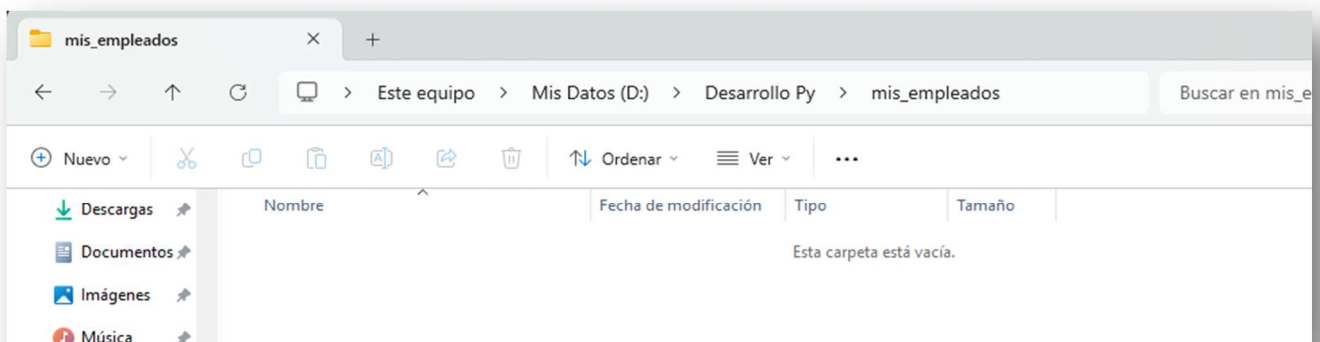
- **venv:** Es una librería integrada en Python 3.3 y versiones posteriores que facilita la creación y gestión de entornos virtuales.
- **virtualenv:** Es otra herramienta popular para crear entornos virtuales, especialmente útil en entornos más complejos.
- **conda:** Es una herramienta de gestión de paquetes y entornos que también permite crear entornos virtuales.

Ejemplo:

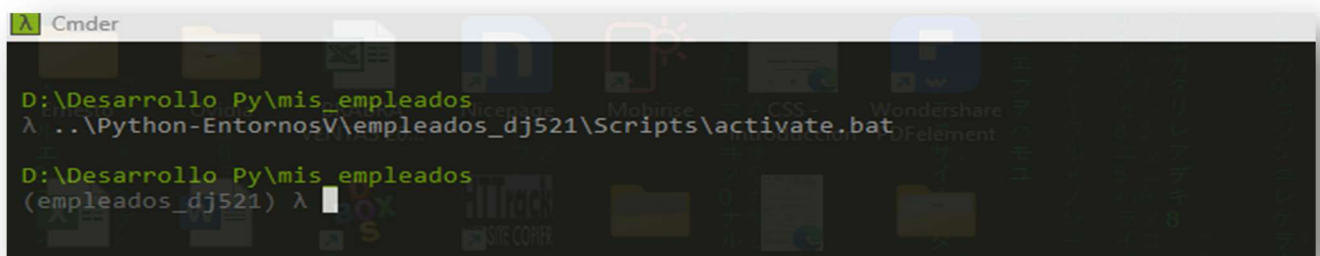
Imagina que tienes dos proyectos, uno que requiere la versión 2.2 de Django y otro que necesita la última versión. Al utilizar entornos virtuales, puedes tener ambas versiones de Django instaladas sin que interfieran entre sí.

En resumen, los entornos virtuales son una herramienta esencial para los desarrolladores de Python que quieren trabajar de manera eficiente y evitar problemas de compatibilidad entre diferentes proyectos.

Continuemos, ingresamos al directorio donde crearemos nuestra aplicación, en este caso la carpeta “**mis_empleados**”, dentro de esa carpeta abrimos nuestro terminal.



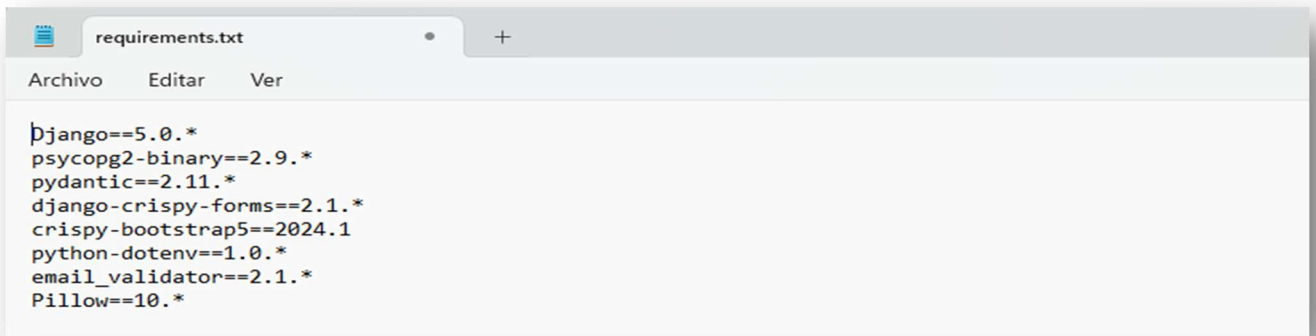
Activamos nuestro entorno virtual : `..\Python-EntornosV\empleados_dj521\Scripts\activate.bat`



INSTALAR DEPENDENCIAS

Para instalar nuestras dependencias de manera automática, es recomendable crear un archivo y especificar cada una de las dependencias que se utilizara en nuestro proyecto. [IA](#)

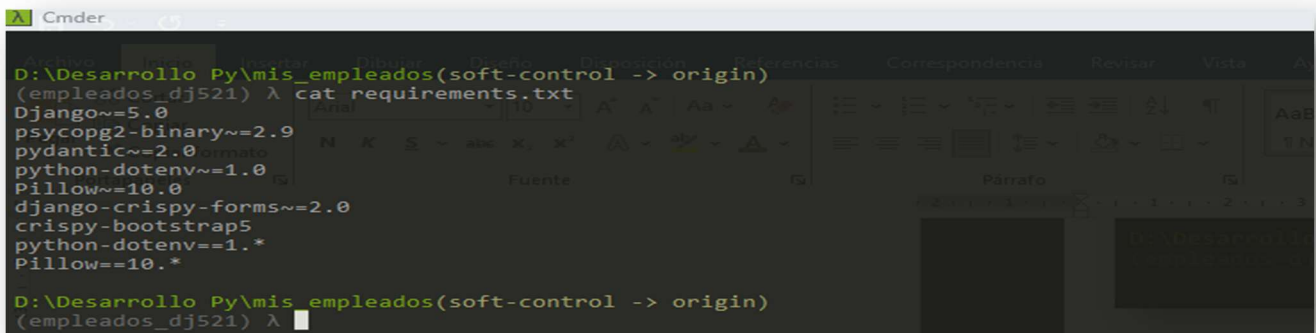
Utilice el editor de su preferencia y cree el archivo `requirements.txt`, Guárdelo en la carpeta `mis_empleados`



```
requirements.txt

Django==5.0.*
psycpg2-binary==2.9.*
pydantic==2.11.*
django-crispy-forms==2.1.*
crispy-bootstrap5==2024.1
python-dotenv==1.0.*
email_validator==2.1.*
Pillow==10.*
```

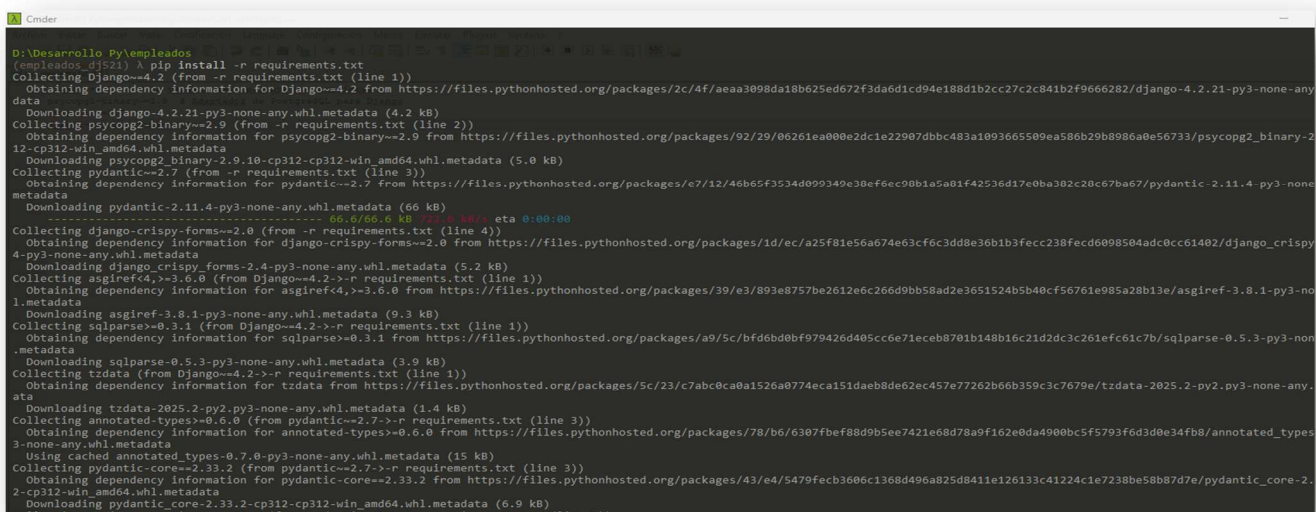
Desde el terminal también puede verificar su contenido : `cat requirements.txt`



```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) > cat requirements.txt
Django==5.0
psycpg2-binary==2.9
pydantic==2.11
python-dotenv==1.0
Pillow==10.0
django-crispy-forms==2.0
crispy-bootstrap5
python-dotenv==1.*
Pillow==10.*

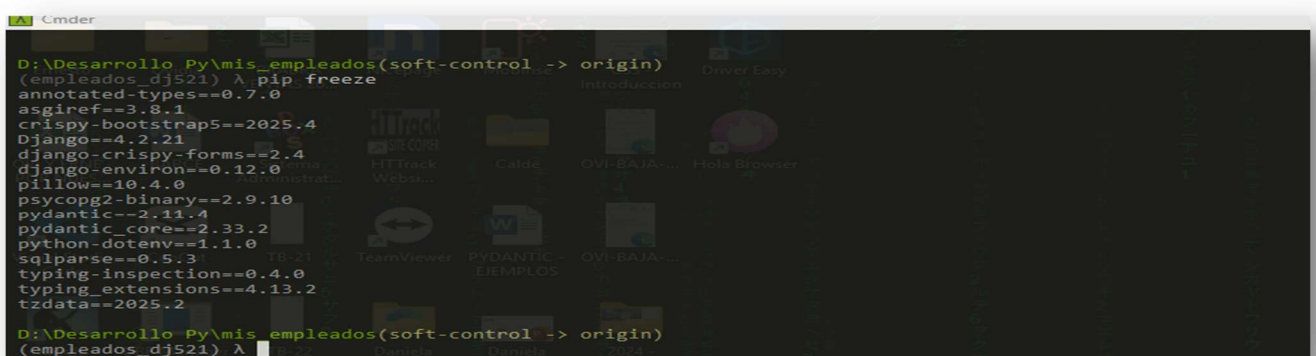
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) >
```

Y procedemos a instalar todas las dependencias : `pip install -r requirements.txt`



```
D:\Desarrollo Py\empleados
(empleados_dj521) > pip install -r requirements.txt
Collecting Django==4.2 (from -r requirements.txt (line 1))
  Obtaining dependency information for Django==4.2 from https://files.pythonhosted.org/packages/2c/4f/aeaa3098da18b625ed672f3da6d1cd94e188db2cc27c2c841b2f9666282/django-4.2.21-py3-none-any.whl
  Downloading django-4.2.21-py3-none-any.whl.metadata (4.2 kB)
Collecting psycpg2-binary==2.9 (from -r requirements.txt (line 2))
  Obtaining dependency information for psycpg2-binary==2.9 from https://files.pythonhosted.org/packages/92/29/06261ea000e2dc1e22907dbbc483a1093665509ea586b29b8986a0e56733/psycpg2_binary-2.9-py3-none-any.whl
  Downloading psycpg2_binary-2.9.10-cp312-cp312-win_amd64.whl.metadata (5.0 kB)
Collecting pydantic==2.7 (from -r requirements.txt (line 3))
  Obtaining dependency information for pydantic==2.7 from https://files.pythonhosted.org/packages/e7/12/46b65f3534d099349e30ef6ec90b1a5a01f42536d17e0ba302c20c67ba67/pydantic-2.11.4-py3-none-any.whl
  Downloading pydantic-2.11.4-py3-none-any.whl.metadata (66 kB)
Collecting django-crispy-forms==2.0 (from -r requirements.txt (line 4))
  Obtaining dependency information for django-crispy-forms==2.0 from https://files.pythonhosted.org/packages/1d/ec/a25f81e56a674e63cf6c3dd8e36b1b3fecc238fec60985a28b13e/django_crispy_forms-2.0-py3-none-any.whl
  Downloading django_crispy_forms-2.4-py3-none-any.whl.metadata (5.2 kB)
Collecting asgiref==4.2 (from -r requirements.txt (line 1))
  Obtaining dependency information for asgiref==4.2 from https://files.pythonhosted.org/packages/39/e3/893e8757be2612e6c266d9bb58ad2e3651524b5b40cf56761e985a28b13e/asgiref-3.8.1-py3-none-any.whl
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse==0.3.1 (from Django==4.2->-r requirements.txt (line 1))
  Obtaining dependency information for sqlparse==0.3.1 from https://files.pythonhosted.org/packages/a9/5c/bfd6bd0bf979426d405cc6e71eceb8701b148b16c21d2dc3c261efc61c7b/sqlparse-0.5.3-py3-none-any.whl
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from Django==4.2->-r requirements.txt (line 1))
  Obtaining dependency information for tzdata from https://files.pythonhosted.org/packages/5c/23/c7abc0ca0a1526a0774eca151daeb8d6e2ec457e77262b66b359c3c7679e/tzdata-2025.2-py2.py3-none-any.whl
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting annotated-types==0.6.0 (from pydantic==2.7->-r requirements.txt (line 3))
  Obtaining dependency information for annotated-types==0.6.0 from https://files.pythonhosted.org/packages/78/b6/6307fbef88d9b5ee7421e68d78a9f162e0da4900bc5f5793f6d3d0e34fb8/annotated_types-0.6.0-py3-none-any.whl
  Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.33.2 (from pydantic==2.7->-r requirements.txt (line 3))
  Obtaining dependency information for pydantic-core==2.33.2 from https://files.pythonhosted.org/packages/43/e4/5479fecb3606c1368d496a825d841e126133c41224c1e7238be58b87d7e/pydantic_core-2.33.2-py3-none-any.whl
  Downloading pydantic_core-2.33.2-cp312-cp312-win_amd64.whl.metadata (6.9 kB)
Collecting typing-extensions==4.12.2 (from pydantic==2.7->-r requirements.txt (line 3))
  Using cached typing_extensions-4.12.2-py3-none-any.whl.metadata (7.1 kB)
```

Verificamos que estén instaladas las dependencias: `pip freeze`



```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) > pip freeze
annotated-types==0.7.0
asgiref==3.8.1
crispy-bootstrap5==2025.4
Django==4.2.21
django-crispy-forms==2.4
django-enviro==0.12.0
pillow==10.4.0
psycpg2-binary==2.9.10
pydantic==2.11.4
pydantic_core==2.33.2
python-dotenv==1.1.0
sqlparse==0.5.3
typing-inspection==0.4.0
typing_extensions==4.12.2
tzdata==2025.2

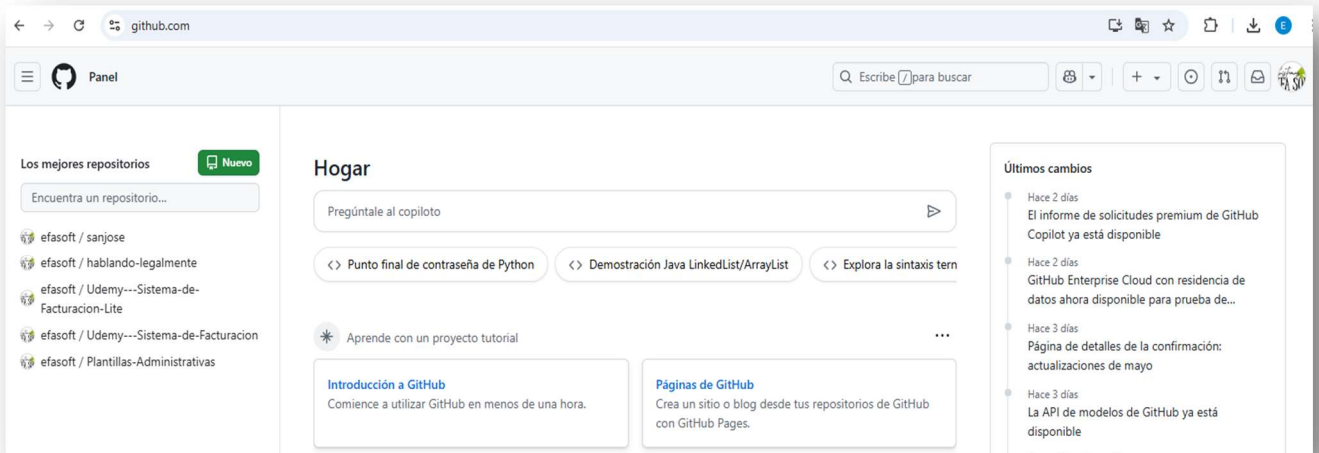
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) >
```


REPOSITORIOS – CONTROL DE VERSIONES

Utilizaremos para el control de versiones la plataforma de **GitHub**. Para ello debemos crear nuestros **repositorios**

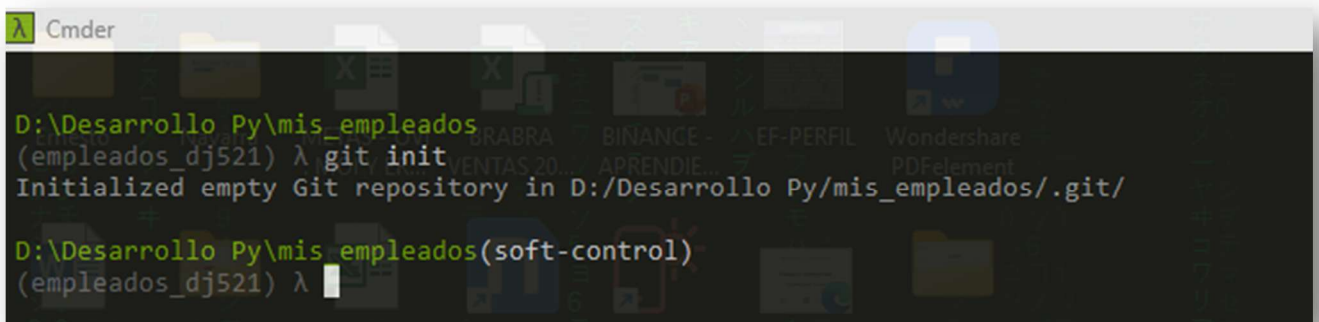
REPOSITORIO REMOTO

Ingresamos en nuestra cuenta de GitHub (<https://github.com/>) y creamos un nuevo repositorio, le pondremos como nombre **empleados**

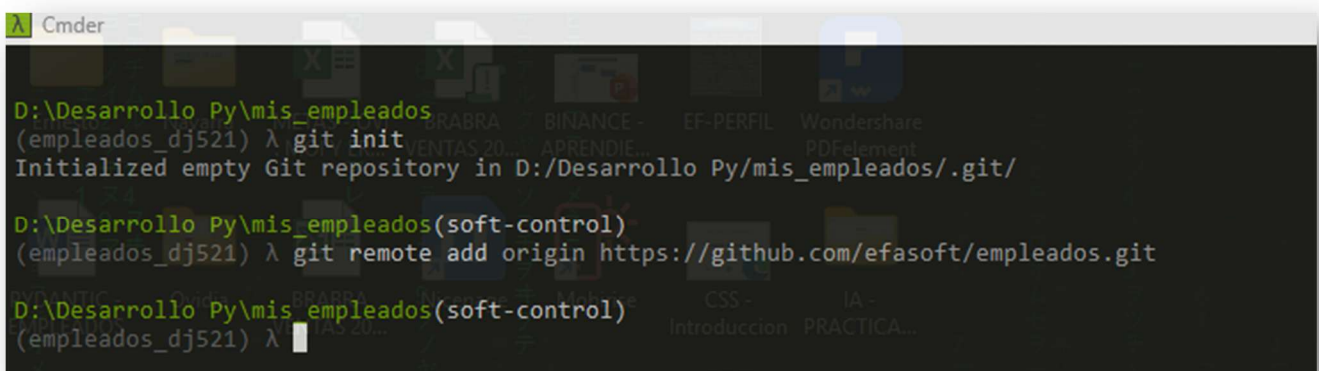


REPOSITORIO LOCAL

Desde el terminal creamos nuestro repositorio local : **git init**



Lo conectamos con nuestro repositorio remoto : **git remote add origin https://github.com/efasoft/empleados.git**

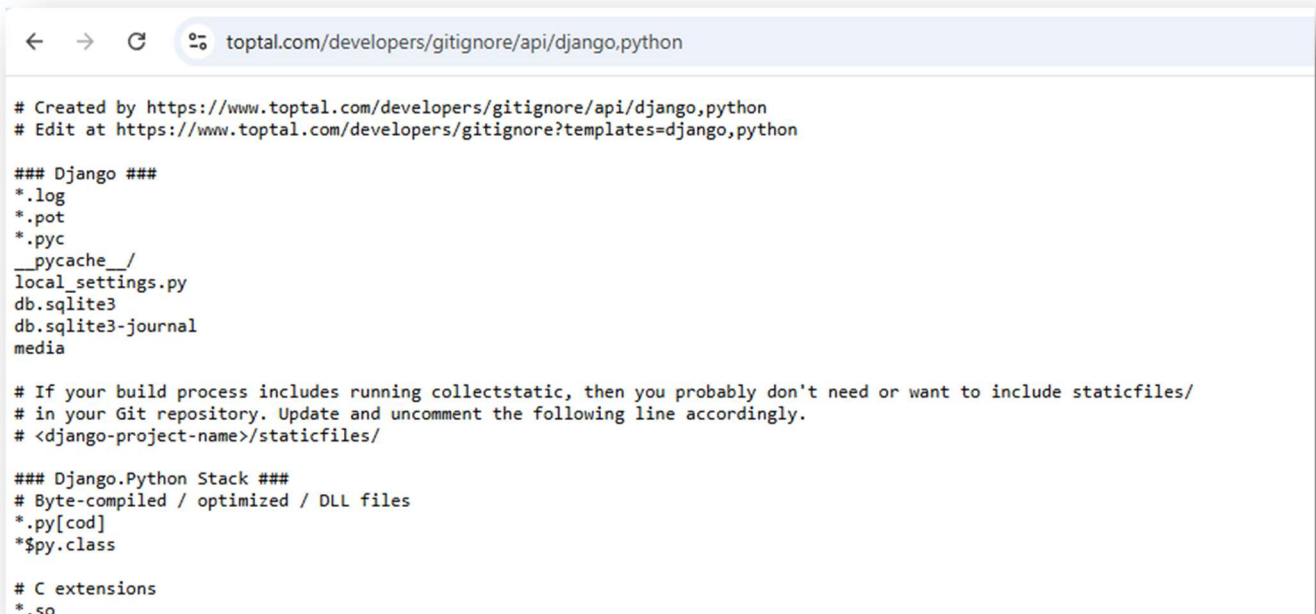


Se recomienda crear el archivo **.gitignore**. El archivo **.gitignore** se utiliza en Git para especificar qué archivos o directorios deben ser ignorados por Git al realizar operaciones como añadir, confirmar o enviar cambios. Esto ayuda a mantener el repositorio limpio y evitar que se incluyan archivos innecesarios o confidenciales en la historia del proyecto.

Para crear el archivo puede ingresar a <https://www.toptal.com/developers/gitignore>

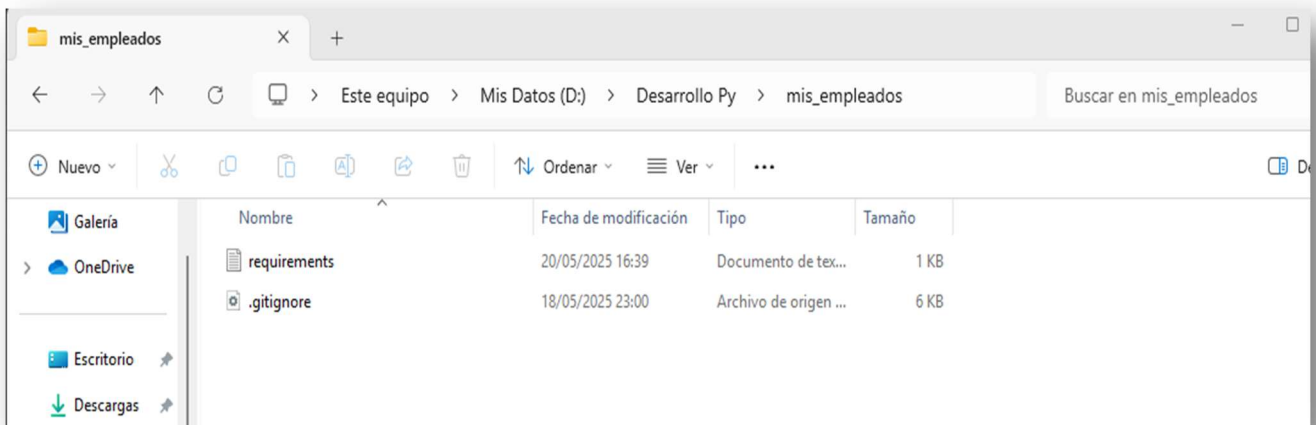


Indique el lenguaje o frameworks que va a utilizar y luego presione el botón . La web le mostrara todo el código generado.



Abra su editor de texto preferido y copie todo el contenido, luego guarde el archivo dentro de la carpeta de su proyecto. (en este caso la carpeta “mis_empleados”). Recuerde el nombre del archivo debe ser .gitignore

Comprobamos



Una vez creado el archivo `.gitignore`, realizamos la primera actualización de nuestros repositorios.

Para agregar todos los archivos que han sido creados o modificados : `git add .`

Identificamos nuestra actualización: `git commit -m "Formulario de Empleados con Django y Pydantic"` (cada vez que haga un `commit` debe poner entre comillas un comentario).

Actualizamos nuestros repositorios: `git push -u origin soft-control`

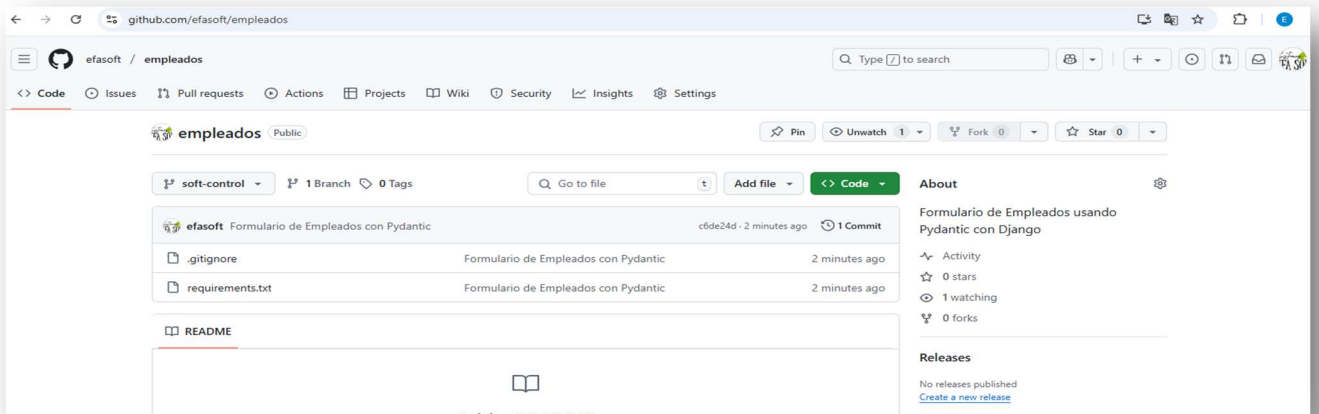
```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git commit -m "Formulario de Empleados con Django y Pydantic"
[soft-control (root-commit) 1aa2b9a] Formulario de Empleados con Django y Pydantic
 2 files changed, 273 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 requirements.txt

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git push -u origin soft-control
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.17 KiB | 2.17 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/efasoft/empleados.git
 * [new branch]      soft-control -> soft-control
branch 'soft-control' set up to track 'origin/soft-control'.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

Comprobamos que nuestro repositorio remoto este actualizado. Recomendamos que periódicamente se actualicen los repositorios.



CONFIGURACIÓN INICIAL DEL PROYECTO

CREACIÓN DEL PROYECTO

Abrimos el terminal dentro de nuestra carpeta de desarrollo y ejecutamos el siguiente comando :

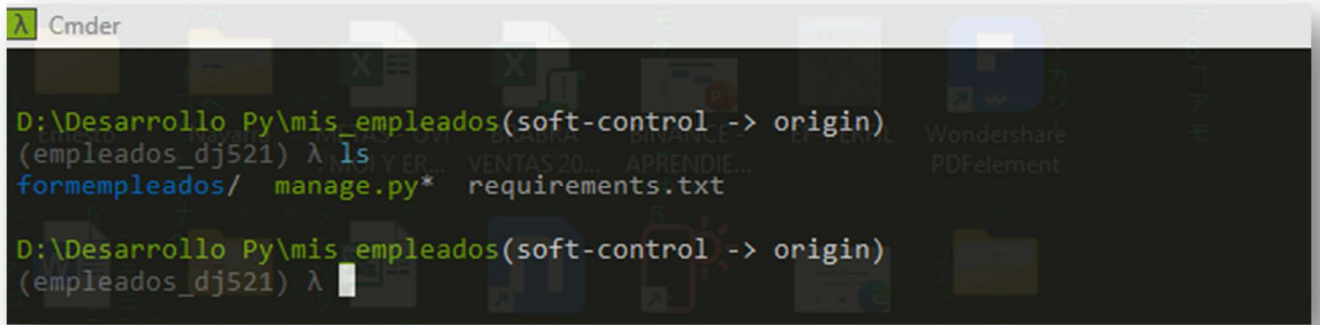
`django-admin.exe startproject formempleados .`

`formempleados .` es el nombre del proyecto, después del nombre deje un espacio y coloque un punto, esto es para que no se cree una carpeta adicional

```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ django-admin.exe startproject formempleados .

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

Verificamos que este creado el proyecto : `ls`



```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ ls
formempleados/  manage.py*  requirements.txt

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

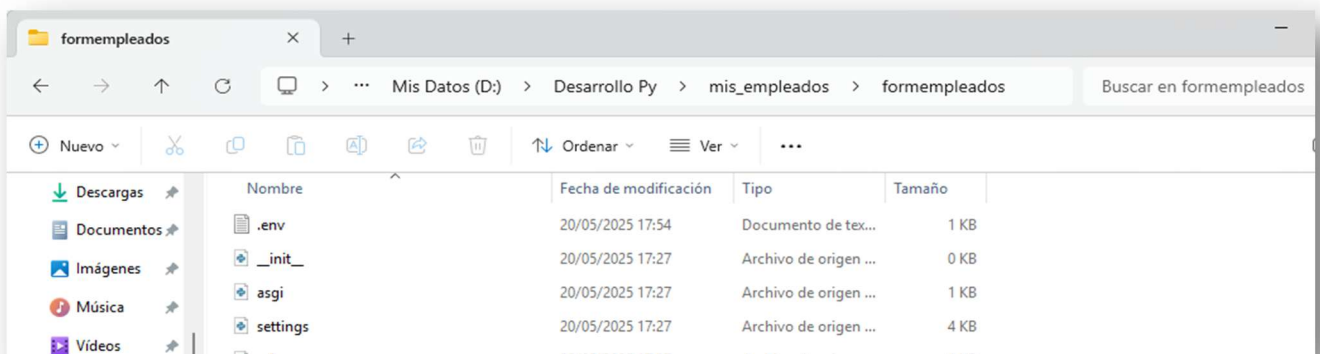
VARIABLES DE ACCESO A LA BASE DE DATOS

Antes de continuar, asegúrate de tener **PostgreSQL** instalado y un usuario y base de datos creados.

Usando nuestro editor preferido `creamos` el archivo `.env` (Este archivo debe estar en la raíz de tu proyecto `form_empleados`)

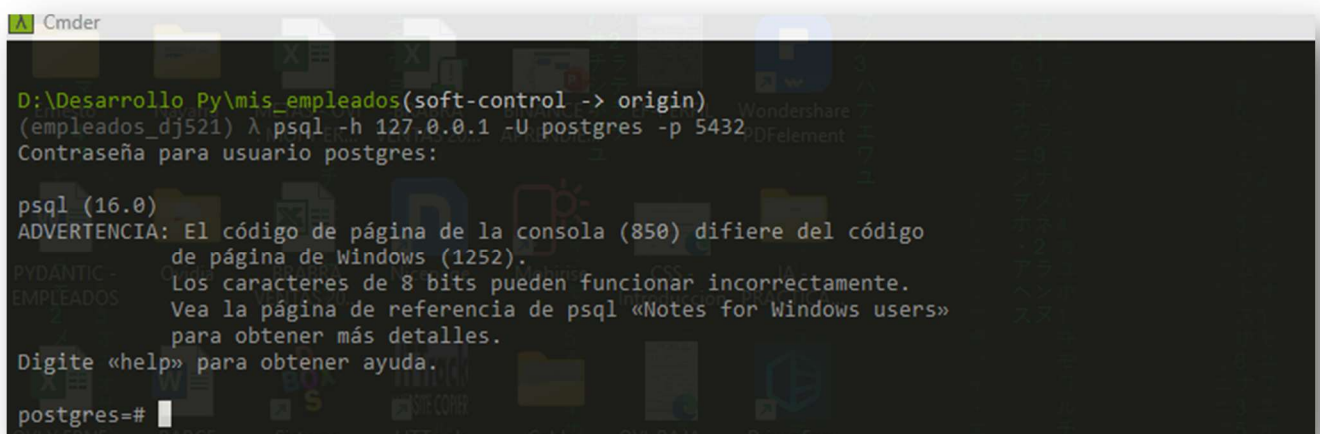
```
DJANGO_SECRET_KEY = keep the secret key used in production secret! (se obtiene desde settings.py)
DB_NAME=your_database
DB_USER=your_postgres_user
DB_PASSWORD=your_postgres_password
DB_HOST=localhost
DB_PORT=5432
```

No olvides modificar y poner los valores reales para acceder a tu Base de Datos



CREAR LA BASE DE DATOS

Desde el terminal nos `conectamos` a PostgreSQL : `psql -h 127.0.0.1 -U postgres -p 5432`



```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ psql -h 127.0.0.1 -U postgres -p 5432
Contraseña para usuario postgres:

psql (16.0)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Digite «help» para obtener ayuda.

postgres=#
```


Explicación:

1. psql:

- psql es la **interfaz de línea de comandos interactiva para PostgreSQL**. Es una aplicación cliente que te permite ejecutar consultas SQL, administrar bases de datos y realizar otras tareas directamente desde tu terminal. Piensa en él como la "consola" o el "terminal" específico para interactuar con tu servidor PostgreSQL.

2. -h 127.0.0.1:

- -h es una opción (flag) que especifica el **nombre de host o la dirección IP del servidor PostgreSQL** al que quieres conectarte.
- 127.0.0.1 es la **dirección IP de "localhost"**. Esto significa que estás intentando conectarte a un servidor PostgreSQL que se está ejecutando en la **misma máquina** donde estás ejecutando el comando psql.

3. -U postgres:

- -U es una opción que especifica el **nombre de usuario** con el que quieres autenticarte en el servidor PostgreSQL.
- postgres es el **nombre de usuario predeterminado** que se crea durante la instalación de PostgreSQL. A menudo se utiliza para tareas de administración y superusuario.

4. -p 5432:

- -p es una opción que especifica el **número de puerto TCP** en el que el servidor PostgreSQL está escuchando las conexiones entrantes.
- 5432 es el **puerto predeterminado** que PostgreSQL utiliza para escuchar las conexiones. A menos que hayas configurado tu servidor PostgreSQL para usar un puerto diferente, este será el puerto correcto.

Creemos la Base de Datos en PostgreSQL : `create database db_empleados_521;`

```
Cmder
D:\Desarrollo_Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ psql -h 127.0.0.1 -U postgres -p 5432
Contraseña para usuario postgres:

psql (16.0)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Digite «help» para obtener ayuda.
postgres=# create database db_empleados_521;
CREATE DATABASE
postgres=#
```

Comprobamos la Base de Datos en PostgreSQL: \l

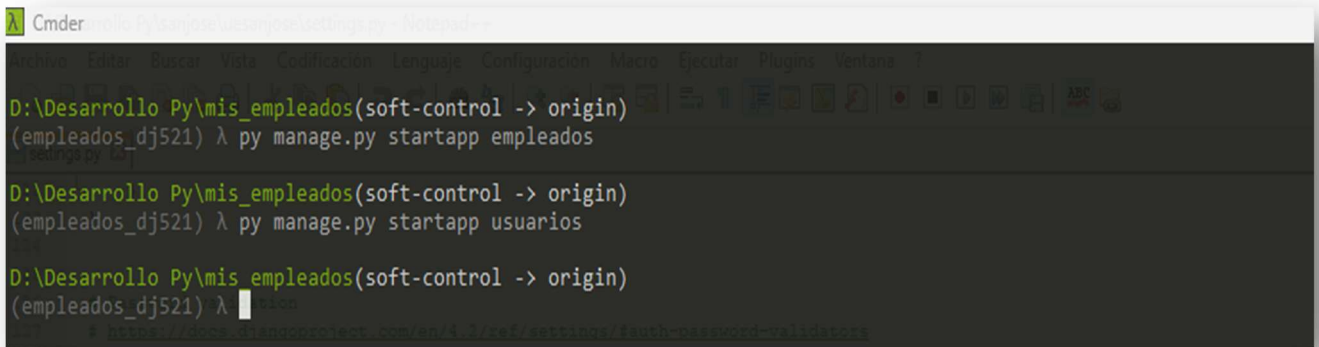
```
postgres=# \l
Listado de base de datos
+-----+-----+-----+-----+-----+-----+-----+
| Nombre | Dueño | Codificación | Proveedor de locale | Collate | Ctype | configuración ICU |
+-----+-----+-----+-----+-----+-----+-----+
| db_djcmpfc_42 | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| db_empleados_521 | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| db_sanjose_42 | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| dbprueba | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| postgres | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| template0 | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
| template1 | postgres | UTF8 | libc | Spanish_Spain.1252 | Spanish_Spain.1252 | |
(7 filas)
postgres=#
```


APP DEL PROYECTO (empleados y usuarios)

Abrimos el terminal dentro de nuestra carpeta de desarrollo y creamos las apps

```
py manage.py startapp empleados
```

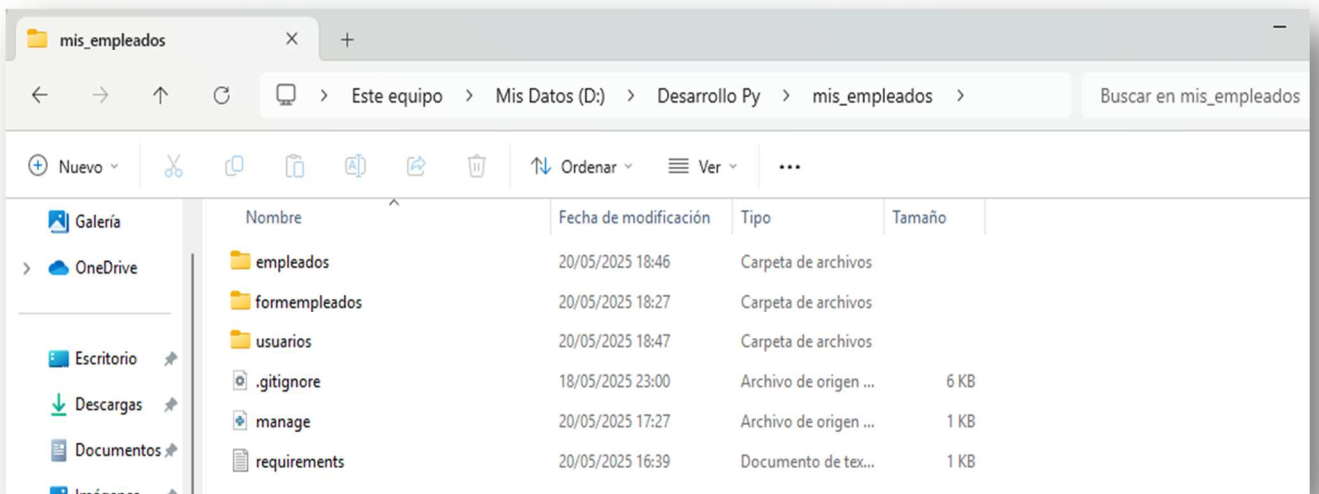
```
py manage.py startapp usuarios
```



```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py startapp empleados

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py startapp usuarios

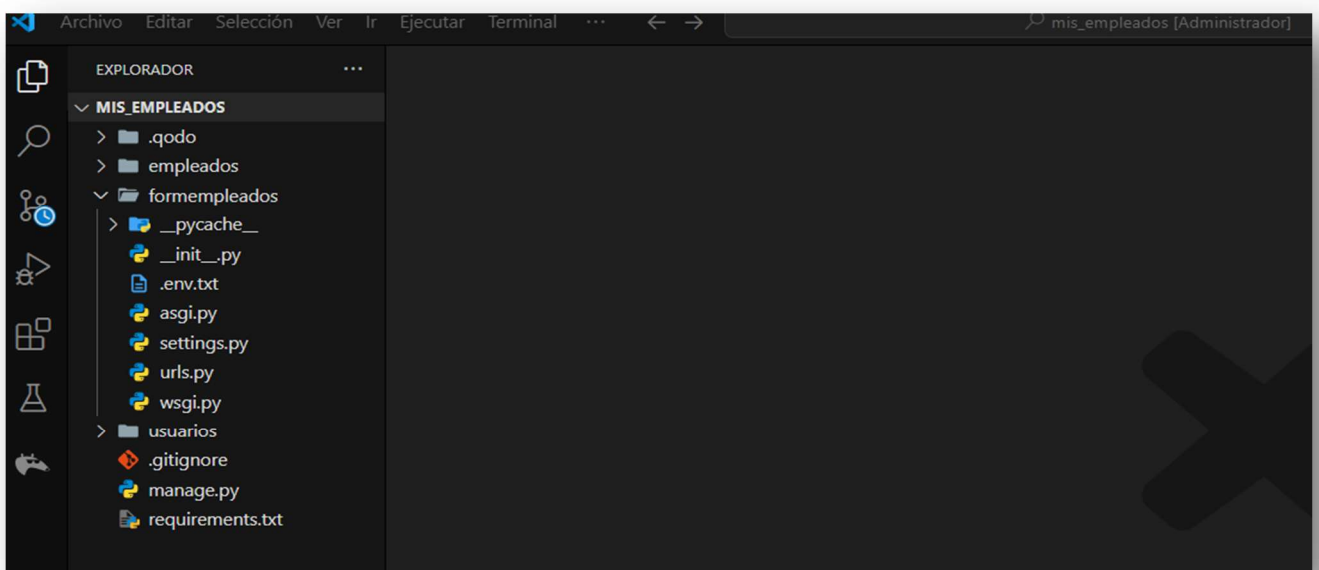
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```



DESARROLLO DEL PROYECTO

VISUAL STUDIO CODE

Desde nuestro terminal abrimos Visual Studio Code : `code -r .`



Observaremos que se crearon las carpetas `formempleados`, `empleados`, `usuarios` y dentro de ellas los archivos iniciales del proyecto.

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'empleados' and 'formempleados', and files like 'settings.py'. The code editor shows the content of 'settings.py', which is a Django settings file generated by 'django-admin startproject' using Django 4.2.21. The file contains various settings for the project, including database configuration, static files, and security settings. A comment at the top right says 'Creado por IA'.

```

1  """
2  Django settings for formempleados project.
3
4  Generated by 'django-admin startproject' using Django 4.2.21.
5
6  |   |   |   Creado por IA
7
8  * from dotenv import load_dotenv
9  * load_dotenv() # Carga las variables de entorno desde .env
10 * SECRET_KEY = os.getenv('DJANGO_SECRET_KEY')
11 * INSTALLED_APPS =
12 * TEMPLATES =
13 * DATABASES =
14 * STATICFILES_DIRS =
15 * MEDIA_URL =
16 * MEDIA_ROOT =
17 * CRISPY_ALLOWED_TEMPLATE_PACKS =
18 * CRISPY_TEMPLATE_PACK =
19 * LOGIN_REDIRECT_URL =
20 * LOGOUT_REDIRECT_URL =
21
22 """
23
24 from pathlib import Path
25 import os

```

Explicación:

- **SECRET_KEY:** Una clave secreta para la seguridad de tu aplicación. ¡Mantenla segura!
- **DEBUG:** Activa o desactiva el modo de depuración. En producción, debe ser False.
- **ALLOWED_HOSTS:** Lista de hosts permitidos para tu aplicación.
- **INSTALLED_APPS:** Las aplicaciones Django que utilizará tu proyecto. Hemos añadido empleados y usuarios.
- **MIDDLEWARE:** Componentes que procesan las peticiones y respuestas.
- **ROOT_URLCONF:** El archivo de URLs principal del proyecto.
- **TEMPLATES:** Configuración de los motores de plantillas.
- **DATABASES:** Configuración de la base de datos PostgreSQL. Usamos environ para leer las variables de entorno.
- **AUTH_PASSWORD_VALIDATORS:** Validadores de contraseñas por defecto de Django.
- **LANGUAGE_CODE y TIME_ZONE:** Configuración de idioma y zona horaria para España.
- **STATIC_URL y STATICFILES_DIRS:** Configuración de archivos estáticos (CSS, JS, imágenes).
- **DEFAULT_AUTO_FIELD:** El tipo de campo automático primario por defecto.
- **MESSAGE_TAGS:** Configuración para que los mensajes de Django se mapeen a los estilos de Bootstrap (para SweetAlert2).
- **AUTH_USER_MODEL:** Indica que usaremos nuestro modelo de usuario personalizado.

MIGRACIONES INICIALES DEL PROYECTO IA

Desde nuestro terminal ejecutamos:

py manage.py makemigrations
py manage.py migrate

The screenshot shows a terminal window with the following commands and output:

```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) > py manage.py makemigrations
No changes detected

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) > py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) >

```

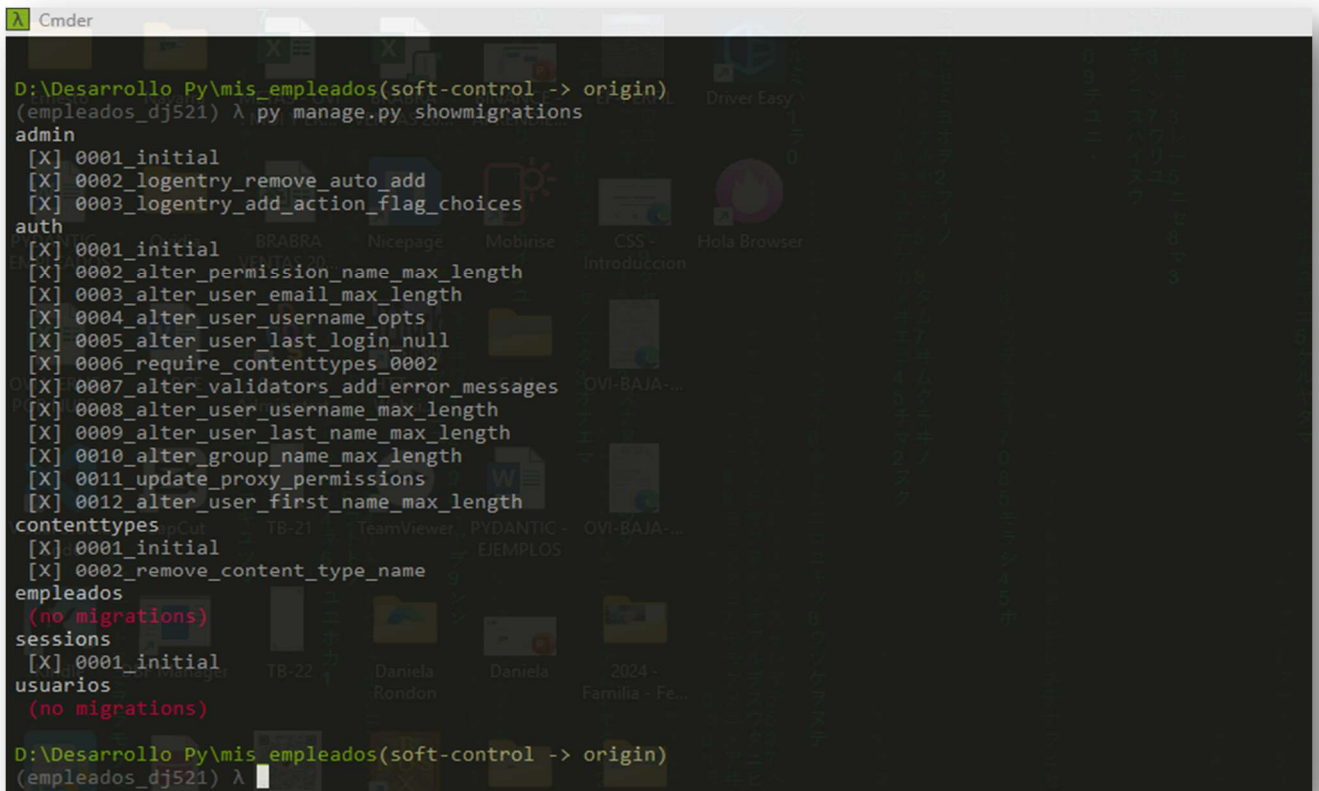
Las migraciones iniciales de `admin`, `auth`, `content types` y `sessions` en Django son un conjunto de tablas y datos que se crean automáticamente al iniciar un proyecto Django. Estas migraciones son fundamentales para el funcionamiento básico del framework, gestionando la interfaz de administración, la autenticación de usuarios, los tipos de contenido y las sesiones de usuario, respectivamente.

Detalle de cada migración:

- **admin:**
Esta migración crea la interfaz de administración de Django, permitiendo a los usuarios gestionar los modelos y datos de la base de datos a través de un panel web.
- **auth:**
Esta migración proporciona la funcionalidad de autenticación y autorización de usuarios. Permite crear usuarios, grupos y permisos para controlar el acceso a diferentes partes de la aplicación.
- **content types:**
Esta migración gestiona la clasificación de los tipos de contenido. Permite identificar las diferentes clases de modelos que se pueden utilizar en Django.
- **sessions:**
Esta migración implementa la gestión de sesiones de usuario. Almacena información temporal sobre la actividad del usuario en la aplicación.

En resumen: estas migraciones iniciales son el punto de partida para el desarrollo de una aplicación Django, proporcionando la base necesaria para la gestión de usuarios, la administración de la aplicación y la gestión de datos. Son creadas automáticamente por Django al ejecutar el comando `makemigrations` por primera vez y se aplican con el comando `migrate`.

Comprobamos la migración inicial : `py manage.py showmigrations`



```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
empleados
(no migrations)
sessions
[X] 0001_initial
usuarios
(no migrations)

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

En este punto del proyecto todavía no hemos definido los datos (en Python llamados “`modelos`”) de las app de `empleados` y `usuarios`, es por eso que al realizar la primera migración se muestra un error en color. Es importante recordar que cada vez que se agregue o modifique un `modelo` se debe realizar el procedimiento de migración.

SUPERUSUARIO – PANEL DE ADMINISTRACIÓN IA

Una vez que levantemos nuestro servidor de desarrollo, podremos ingresar al Panel de Administración de Django, usando un `superusuario`. Para poder crearlo ingresamos a nuestro terminal y `ejecutamos`:

`py manage.py createsuperuser`

```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py createsuperuser
Nombre de usuario (leave blank to use 'ernesto'):
```

Dirección de correo electrónico: efasoft@hotmail.com

Password:

Password (again):

Superuser created successfully.

```
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

Para este proyecto el **password** será: **empleados2025**

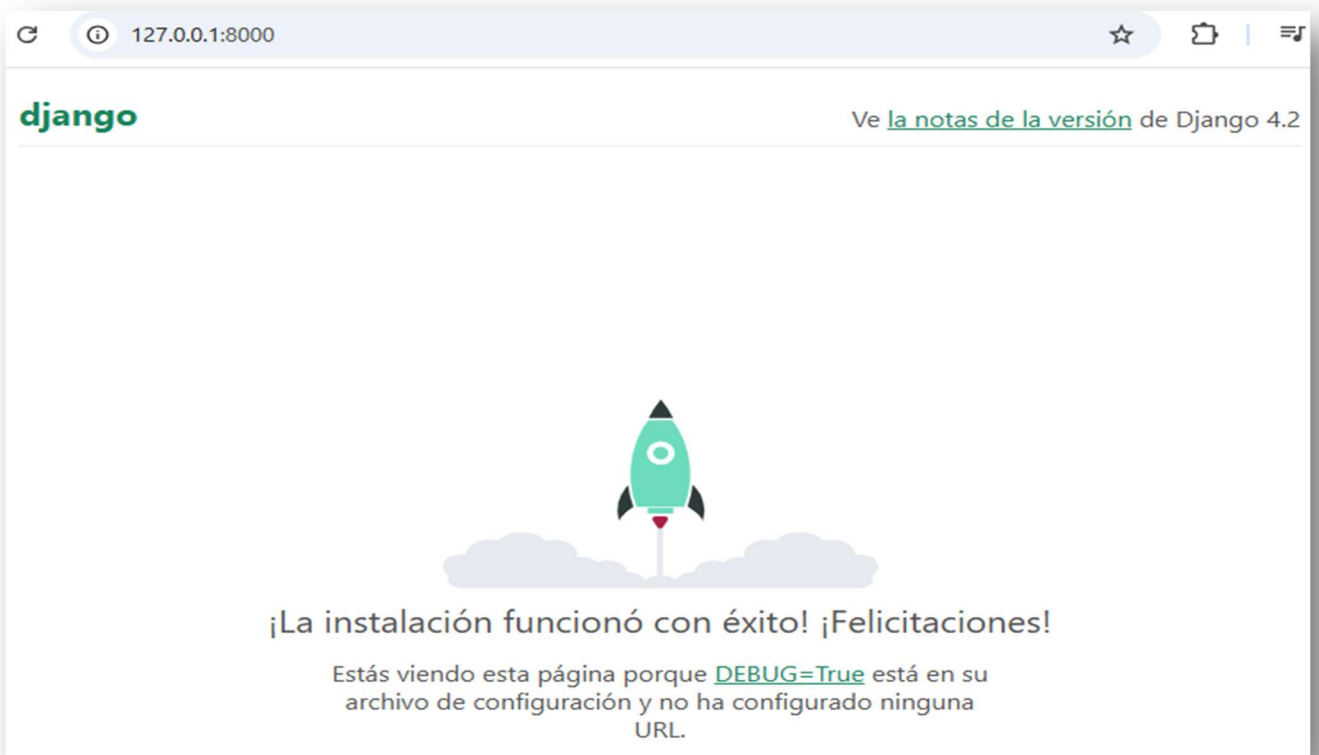
SERVIDOR DE DESARROLLO IA

Una vez terminada la configuración inicial del proyecto, verificamos que todo funcione. Para ello **levantamos** nuestro servidor de desarrollo : **py manage.py runserver**

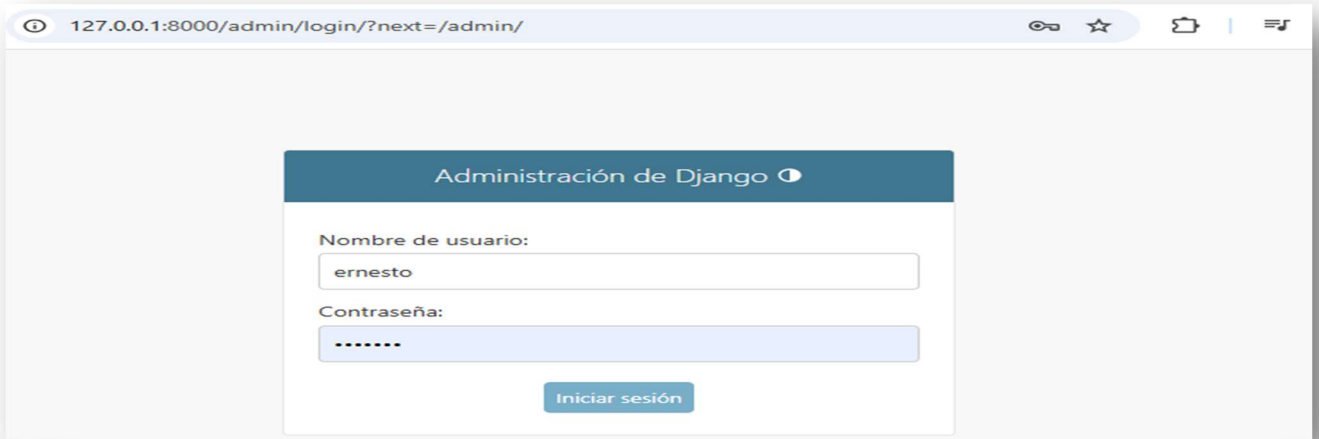
```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 22, 2025 - 19:44:23
Django version 4.2.21, using settings 'formempleados.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

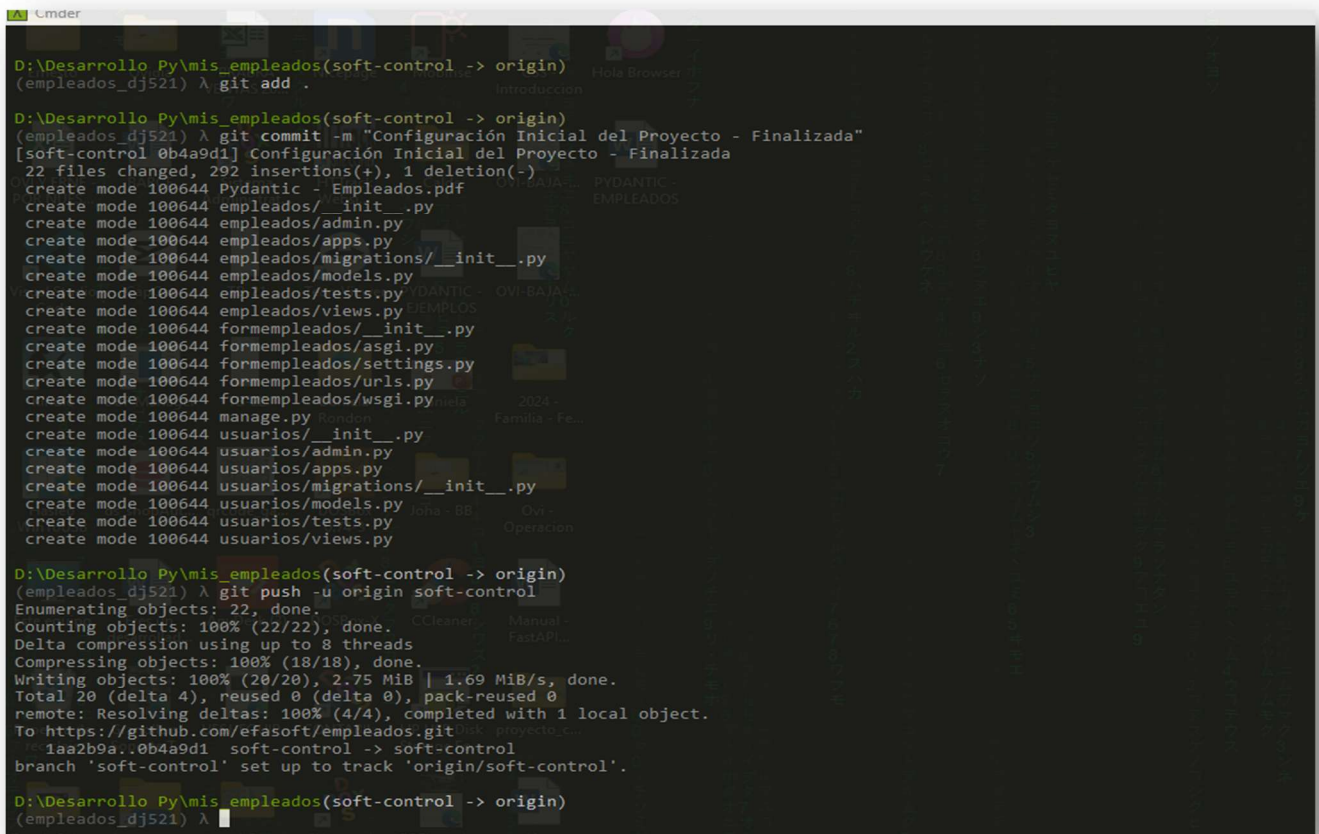
Abre tu navegador preferido e ingresa a **http://127.0.0.1:8000/**

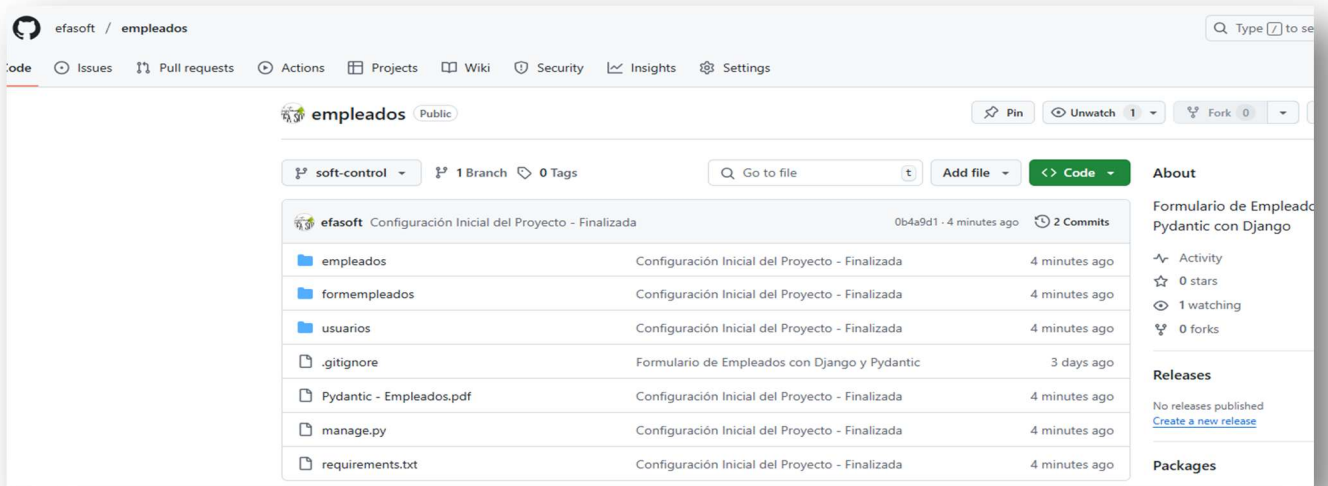


Para ingresar al Panel de Administración, ingresa a <http://127.0.0.1:8000/admin/>.



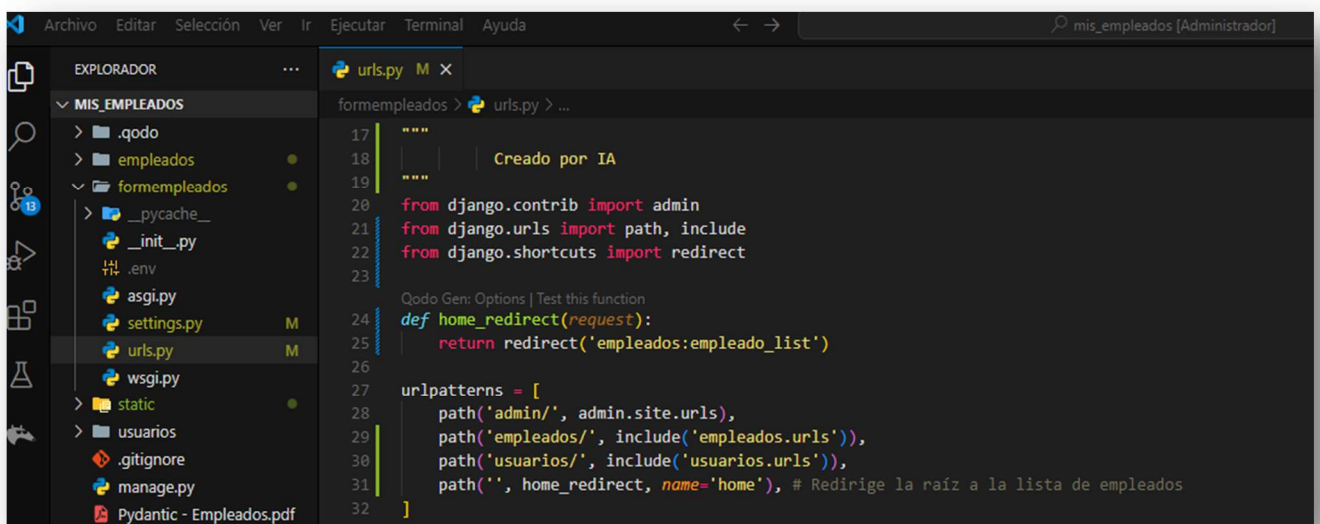
ACTUALIZAMOS NUESTRO REPOSITORIOS





CONTENIDO DE LOS ARCHIVOS DE DESARROLLO

RUTA PRINCIPAL `urls.py` IA



Se establecen las rutas **URLs** principales del proyecto, incluyendo las URLs de la aplicación **empleados** y **usuarios**.

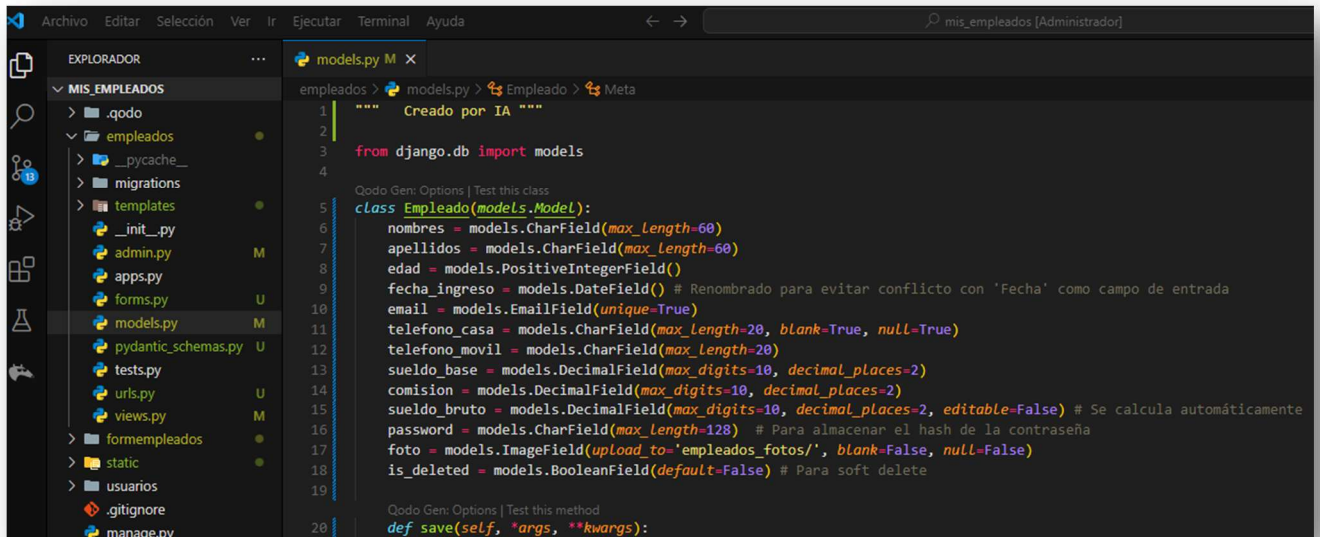
En el contexto de Django, **urls.py** es un archivo que define cómo se mapean las URLs (direcciones web) a las vistas (funciones de Python que manejan las solicitudes HTTP) dentro de tu aplicación o proyecto. Es como una tabla de contenido para tu web, que indica qué debe ocurrir cuando el usuario visita una determinada dirección.

En detalle:

- **Mapeo de URLs:**
urls.py contiene una lista de patrones de URL que Django usará para determinar qué vista debe ejecutarse para una solicitud dada.
- **Vistas:**
Las vistas son funciones de Python que reciben una solicitud HTTP y devuelven una respuesta HTTP, como una página HTML.
- **Patrones de URL:**
Se definen usando la función `path()` o `re_path()` en urls.py. Estos patrones especifican una parte de la URL que se debe coincidir y la vista que se debe ejecutar si se encuentra una coincidencia.
- **Enrutamiento:**
El proceso de encontrar la vista correcta para una URL se llama enrutamiento, y urls.py es fundamental para este proceso.
- **Ejemplo:**
Si tienes una URL `/home/` definida en urls.py y una vista llamada `home` que procesa solicitudes a esa URL, cuando un usuario visita `/home/`, Django buscará en urls.py, encontrará el patrón de URL correspondiente y llamará a la vista `home`.

APP EMPLEADOS

ESTRUCTURA DE DATOS `models.py` IA



```
1 """ Creado por IA """
2
3 from django.db import models
4
5 class Empleado(models.Model):
6     nombres = models.CharField(max_length=60)
7     apellidos = models.CharField(max_length=60)
8     edad = models.PositiveIntegerField()
9     fecha_ingreso = models.DateField() # Renombrado para evitar conflicto con 'Fecha' como campo de entrada
10    email = models.EmailField(unique=True)
11    telefono_casa = models.CharField(max_length=20, blank=True, null=True)
12    telefono_movil = models.CharField(max_length=20)
13    sueldo_base = models.DecimalField(max_digits=10, decimal_places=2)
14    comision = models.DecimalField(max_digits=10, decimal_places=2)
15    sueldo_bruto = models.DecimalField(max_digits=10, decimal_places=2, editable=False) # Se calcula automáticamente
16    password = models.CharField(max_length=128) # Para almacenar el hash de la contraseña
17    foto = models.ImageField(upload_to='empleados_fotos/', blank=False, null=False)
18    is_deleted = models.BooleanField(default=False) # Para soft delete
19
20 def save(self, *args, **kwargs):
```

Se definen los campos de la tabla `empleados`

En el contexto de Django, `models.py` es un archivo fundamental que define los modelos de datos de tu aplicación. Estos modelos, que son clases Python, describen la estructura de los datos que se almacenarán en la base de datos y proporcionan herramientas para interactuar con ellos.

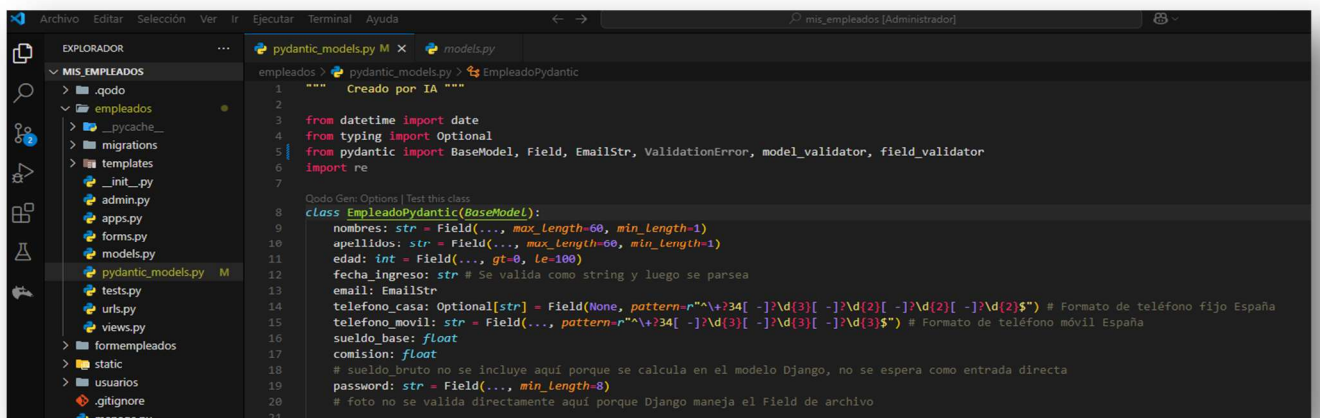
En detalle:

- **Definición de la estructura de datos:**
En `models.py`, defines los campos de tus modelos (ej. `CharField` para texto, `IntegerField` para números, etc.) y sus propiedades (ej. longitud máxima, valores predeterminados).
- **Relaciones entre modelos:**
Puedes definir relaciones entre diferentes modelos (ej. uno a uno, uno a muchos, muchos a muchos) utilizando los campos adecuados.
- **Manipulación de datos:**
Django proporciona métodos para crear, leer, actualizar y eliminar (CRUD) datos a través de los modelos.
- **Creación de tablas en la base de datos:**
Al ejecutar `makemigrations` y `migrate`, Django crea las tablas en la base de datos basándose en los modelos definidos en `models.py`.

Ejemplo:

Supongamos que quieres modelar un artículo de blog. Podrías definir un modelo `BlogEntry` con campos como `title` (texto), `content` (texto largo), `pub_date` (fecha), etc., y luego usar ese modelo para crear, modificar, y consultar entradas de blog en la base de datos.

VALIDACIONES `pydantic_models.py` IA



```
1 """ Creado por IA """
2
3 from datetime import date
4 from typing import Optional
5 from pydantic import BaseModel, Field, EmailStr, ValidationError, model_validator, field_validator
6 import re
7
8 class EmpleadoPydantic(BaseModel):
9     nombres: str = Field(..., max_length=60, min_length=1)
10    apellidos: str = Field(..., max_length=60, min_length=1)
11    edad: int = Field(..., gt=0, le=100)
12    fecha_ingreso: str # Se valida como string y luego se parsea
13    email: EmailStr
14    telefono_casa: Optional[str] = Field(None, pattern=r"^\+34[ -]\d{3}[ -]\d{2}[ -]\d{2}$") # Formato de teléfono fijo España
15    telefono_movil: str = Field(..., pattern=r"^\+34[ -]\d{3}[ -]\d{3}[ -]\d{2}$") # Formato de teléfono móvil España
16    sueldo_base: float
17    comision: float
18    # sueldo_bruto no se incluye aquí porque se calcula en el modelo Django, no se espera como entrada directa
19    password: str = Field(..., min_length=8)
20    # foto no se valida directamente aquí porque Django maneja el Field de archivo
21
```

Se establecen las funciones con **pydantic** para **validar** cada campos de la tabla **empleados**

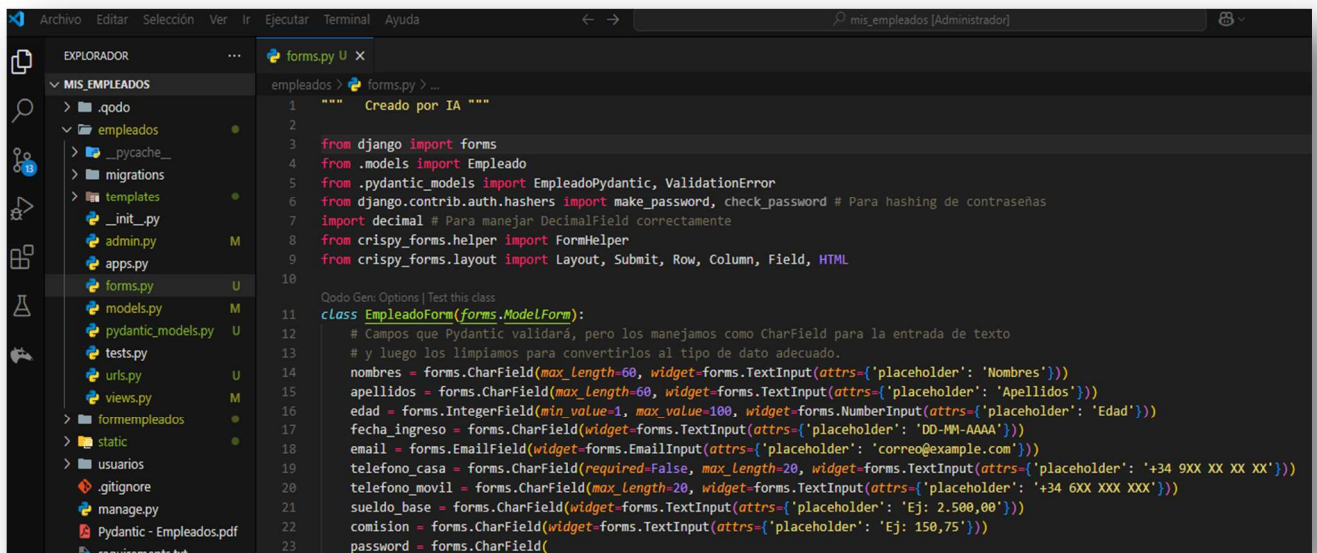
¿Qué es Pydantic?

- Pydantic es una biblioteca que facilita la creación de estructuras de datos que pueden ser utilizadas para la validación, serialización y deserialización de datos.
- Permite definir clases que representan la estructura de los datos que se espera recibir (por ejemplo, desde una API, un formulario o un archivo de configuración).
- Estas clases utilizan anotaciones de tipo de Python para indicar el tipo de datos esperado para cada atributo.
- Pydantic valida los datos que se pasan a estas clases, asegurando que coincidan con las anotaciones de tipo definidas, según Pydantic.

¿Por qué se utiliza?

- **Validación:**
Pydantic verifica que los datos que se pasan a una clase coincidan con la estructura y el tipo esperados, evitando errores inesperados.
- **Gestión de configuraciones:**
Facilita la definición de esquemas de configuración y la validación de los valores que se obtienen de un archivo de configuración.
- **Serialización/Deserialización:**
Permite convertir fácilmente entre objetos Python y formatos como JSON, facilitando la interoperabilidad con otras herramientas.

FORMULARIO **forms.py** IA



```
1 """ Creado por IA """
2
3 from django import forms
4 from .models import Empleado
5 from pydantic_models import EmpleadoPydantic, ValidationError
6 from django.contrib.auth.hashers import make_password, check_password # Para hashing de contraseñas
7 import decimal # Para manejar DecimalField correctamente
8 from crispy_forms.helper import FormHelper
9 from crispy_forms.layout import Layout, Submit, Row, Column, Field, HTML
10
11 Qodo Gen: Options | Test this class
12 class EmpleadoForm(forms.ModelForm):
13     # Campos que Pydantic validará, pero los manejamos como CharField para la entrada de texto
14     # y luego los limpiamos para convertirlos al tipo de dato adecuado.
15     nombres = forms.CharField(max_length=60, widget=forms.TextInput(attrs={'placeholder': 'Nombres'}))
16     apellidos = forms.CharField(max_length=60, widget=forms.TextInput(attrs={'placeholder': 'Apellidos'}))
17     edad = forms.IntegerField(min_value=1, max_value=100, widget=forms.NumberInput(attrs={'placeholder': 'Edad'}))
18     fecha_ingreso = forms.CharField(widget=forms.TextInput(attrs={'placeholder': 'DD-MM-AAAA'}))
19     email = forms.EmailField(widget=forms.EmailInput(attrs={'placeholder': 'correo@example.com'}))
20     telefono_casa = forms.CharField(required=False, max_length=20, widget=forms.TextInput(attrs={'placeholder': '+34 9XX XX XX XX'}))
21     telefono_movil = forms.CharField(max_length=20, widget=forms.TextInput(attrs={'placeholder': '+34 6XX XXX XXX'}))
22     sueldo_base = forms.CharField(widget=forms.TextInput(attrs={'placeholder': 'Ej: 2.500,00'}))
23     comision = forms.CharField(widget=forms.TextInput(attrs={'placeholder': 'Ej: 150,75'}))
24     password = forms.CharField(
```

En Django, **forms.py** es un archivo donde se define la lógica para crear formularios. Es una convención, no una obligación, para organizar el código de formularios dentro de una aplicación. Aquí hay algunos detalles:

- **¿Para qué sirve?**
forms.py sirve para:
 - Definir la estructura de los formularios, incluyendo los campos, validaciones y comportamientos.
 - Crear clases de formularios, como ModelForm para conectar los formularios con los modelos de datos de Django.
 - Facilitar la gestión y mantenimiento del código de los formularios.
- **¿Por qué usarlo?**
Organizar el código de formularios en un archivo separado mejora la estructura del proyecto, lo que facilita la lectura, la modificación y el mantenimiento del código.
- **¿Cómo funciona?**
En Django, puedes usar clases de formulario para definir la estructura de un formulario, especificar qué campos deben tener y cómo se validan. Un ModelForm se usa para conectar un formulario a un modelo de datos, permitiendo crear, editar y eliminar objetos del modelo a través de un formulario.

```

1  """ Creado por IA """
2
3  from django.shortcuts import render, get_object_or_404, redirect
4  from django.views.generic import ListView, CreateView, UpdateView, DeleteView
5  from django.urls import reverse_lazy
6  from django.contrib.auth.mixins import LoginRequiredMixin
7  from .models import Empleado
8  from .forms import EmpleadoForm
9  from django.contrib import messages
10 from django.http import JsonResponse
11 from django.views import View
12
13 Qodo Gen: Options | Test this class
14 class EmpleadoListView(LoginRequiredMixin, ListView):
15     model = Empleado
16     template_name = 'empleados/empleado_list.html'
17     context_object_name = 'empleados'
18     paginate_by = 10 # Paginación
19
20 Qodo Gen: Options | Test this method
21 def get_queryset(self):
22     return Empleado.objects.filter(is_deleted=False) # Solo mostrar empleados no eliminados
23
24 Qodo Gen: Options | Test this class
25 class EmpleadoCreateView(LoginRequiredMixin, CreateView):

```

En el contexto de Django, `views.py` es un archivo en Python donde se define la lógica que maneja las solicitudes y respuestas de una aplicación web. Es un componente esencial del modelo MTV ([Modelo-Plantilla-Vista](#)) de Django, donde las vistas se encargan de procesar la lógica de la aplicación y devolver una respuesta al usuario, como una página HTML.

En detalle:

- **Lógica de la aplicación:**

Las vistas contienen el código Python que realiza acciones como consultar datos de la base de datos, procesar formularios, y devolver respuestas.

- **Solicitudes y respuestas:**

Las vistas reciben solicitudes HTTP del usuario y devuelven respuestas HTTP, como páginas HTML, JSON o datos de API.

- **Organización:**

Es común que las vistas de una aplicación se organicen en el archivo `views.py` dentro de la carpeta de la aplicación.

- **URL mapping:**

Las vistas se mapean a URLs a través de un archivo `urls.py`, que establece cómo Django debe dirigir las solicitudes entrantes a las vistas correctas, según explica Django Girls.

- **Tipos de vistas:**

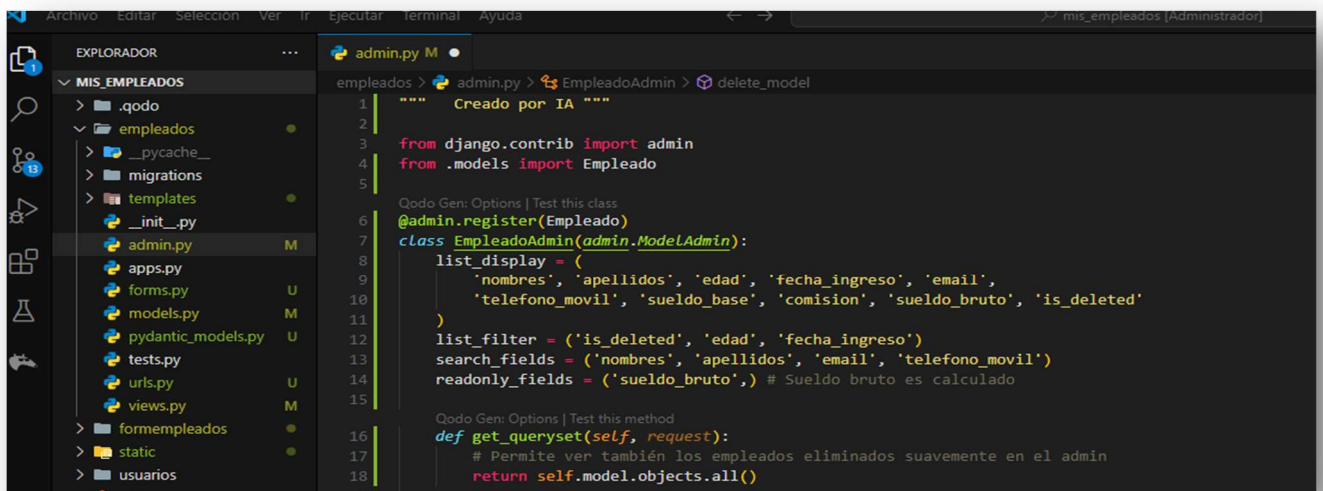
Django ofrece tanto vistas basadas en funciones como vistas basadas en clases, que permiten organizar la lógica de la aplicación de diferentes maneras.

```

1  """ Creado por IA """
2
3  > from django.urls import path...
4
5  app_name = 'empleados'
6
7  urlpatterns = [
8
9      path('', EmpleadoListView.as_view(), name='empleado_list'),
10     path('crear/', EmpleadoCreateView.as_view(), name='empleado_create'),
11     path('editar/<int:pk>/', EmpleadoUpdateView.as_view(), name='empleado_update'),
12     path('eliminar/<int:pk>/', EmpleadoDeleteView.as_view(), name='empleado_delete'),
13 ]

```


REGISTRAR EL MODELO EN EL PANEL DE ADMINISTRACIÓN admin.py IA



admin.py es un archivo de Python dentro de una aplicación Django que se utiliza para registrar los modelos de la aplicación en el **panel de administración** de Django. Este panel de administración permite gestionar los datos de la base de datos de manera sencilla a través de una interfaz web. Al registrar los modelos en **admin.py**, Django genera automáticamente formularios y vistas para crear, leer, actualizar y eliminar (CRUD) los datos de los modelos, facilitando así la gestión de la base de datos.

En resumen, **admin.py** es el archivo que conecta tus modelos de datos con el panel de administración de Django, permitiéndote gestionar los datos de tu aplicación de forma intuitiva y eficiente. Para acceder a ella debemos ir a <http://127.0.0.1:8000/admin/>.

Importante : No es obligatorio en un proyecto tener nuestros modelos en el Panel de Administración

Detalles adicionales:

- **Ubicación:**

admin.py se encuentra dentro de la carpeta de la aplicación, junto con otros archivos como **models.py**, **views.py**, etc.

- **Función:**

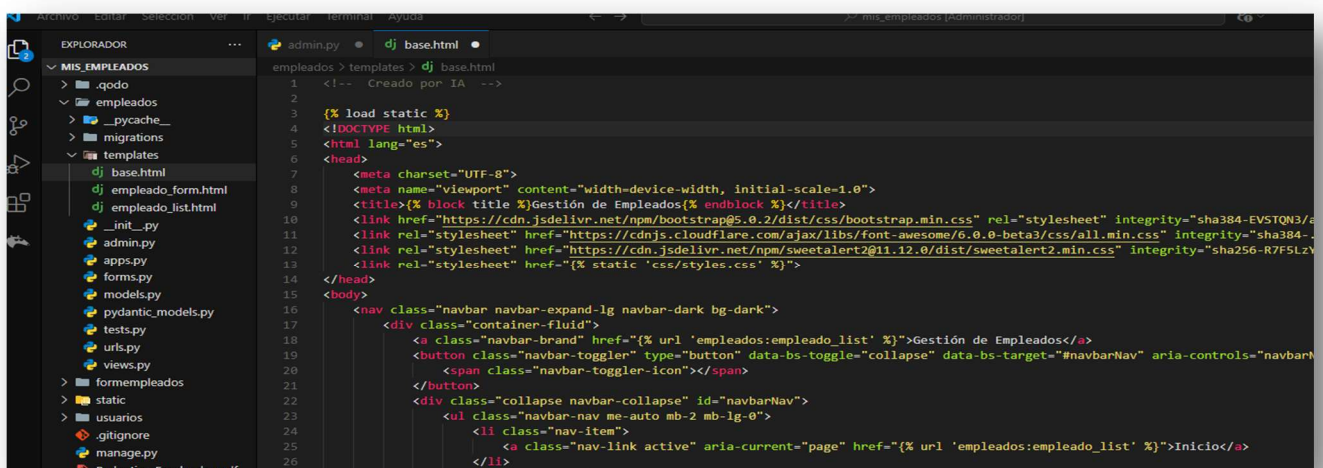
Al registrar un modelo en **admin.py**, puedes personalizar la interfaz de gestión, incluyendo la ordenación de campos, la agrupación de registros, etc.

- **Beneficios:**

Facilita la gestión de datos, proporciona una interfaz web intuitiva, permite personalizar la experiencia de administración, y es una herramienta poderosa para la administración de aplicaciones Django.

TEMPLATES

GESTIÓN DE EMPLEADOS base.html IA



LISTAR EMPLEADOS empleado_list.html IA

```

EXPLORADOR
MIS_EMPLEADOS
  empleados
  migrations
  _pycache_
  __init__.py
  0001_initial.py
  templates
    dj_base.html
    dj_empleado_confirm_dele...
    dj_empleado_form.html
    dj_empleado_list.html
    dj_password_input.html
    __init__.py
    admin.py
    apps.py
    forms.py
    models.py
    pydantic_models.py
    tests.py
    urls.py
    views.py
  formempleados
  _pycache_
  __init__.py
  .env

dj empleado_list.html M X
empleados > templates > dj empleado_list.html
1 <!-- Creado por IA -->
2
3 {% extends 'base.html' %}
4 {% load static %}
5
6 {% block title %}Lista de Empleados{% endblock %}
7
8 {% block content %}
9 <div class="d-flex justify-content-between align-items-center mb-4">
10 <h2>Lista de Empleados</h2>
11 <a href="{% url 'empleados:empleado_create' %}" class="btn btn-success">
12 <i class="fas fa-plus-circle me-2"></i>Crear Nuevo Empleado
13 </a>
14 </div>
15
16 {% if empleados %}
17 <div class="table-responsive">
18 <table class="table table-striped table-hover">
19 <thead>
20 <tr>
21 <th>Nombres</th>
22 <th>Apellidos</th>
23 <th>Edad</th>
24 <th>Email</th>
25 <th>Teléfono Móvil</th>
26 <th>Sueldo Bruto</th>
27 <th>Acciones</th>
28 </tr>

```

REGISTRO DE DATOS EMPLEADOS empleado_form.html IA

```

Archivo Editar Selección Ver Ir Ejecutar Terminal Ayuda mis_empleados [Administrador]

EXPLORADOR
MIS_EMPLEADOS
  .qodo
  empleados
  _pycache_
  migrations
  templates
    dj_base.html
    dj_empleado_form.html
    dj_empleado_list.html
    __init__.py
    admin.py
    apps.py
    forms.py
    models.py
    pydantic_models.py
    tests.py
    urls.py

dj empleado_form.html U X
empleados > templates > dj empleado_form.html
1 <!-- Creado por IA -->
2
3 {% extends 'empleados/base.html' %}
4 {% load crispy_forms_tags %}
5 {% load static %}
6
7 {% block title %}{% if form.instance.pk %}Editar Empleado{% else %}Crear Empleado{% endif %}{% endblock %}
8
9 {% block content %}
10 <div class="card shadow-sm">
11 <div class="card-header bg-primary text-white">
12 <h4 class="mb-0">{% if form.instance.pk %}Editar Empleado{% else %}Crear Nuevo Empleado{% endif %}</h4>
13 </div>
14 <div class="card-body">
15 <form method="post" enctype="multipart/form-data" class="form-horizontal">
16 <div class="form-group">
17 <div class="form-text">{{ csrf_token }}
18 </div>
19 <div class="form-text">{{ form.crispy }}
20 </div>
21 </form>
22 </div>

```

ELIMINAR DATOS EMPLEADOS empleado_confirm_delete.html IA

```

Archivo Editar Selección Ver Ir Ejecutar Terminal Ayuda mis_empleados [Administrador]

EXPLORADOR
MIS_EMPLEADOS
  .qodo
  empleados
  _pycache_
  migrations
  templates
    dj_base.html
    dj_empleado_confirm...
    dj_empleado_form.html
    dj_empleado_list.html
    __init__.py
    admin.py
    apps.py
    forms.py
    models.py
    pydantic_models.py
    tests.py
    urls.py
    views.py
  formempleados

dj empleado_confirm_delete.html U X
empleados > templates > dj empleado_confirm_delete.html
1 <!-- Creado por IA -->
2
3 {% extends 'empleados/base.html' %}
4 {% load static %}
5
6 {% block title %}Confirmar Eliminación{% endblock %}
7
8 {% block content %}
9 <div class="card shadow-sm">
10 <div class="card-header bg-danger text-white">
11 <h4 class="mb-0">Confirmar Eliminación</h4>
12 </div>
13 <div class="card-body">
14 <p class="lead">Estás seguro de que quieres eliminar a **{{ object.nombres }} {{ object.apellidos }}**</p>
15 <div class="alert alert-warning" role="alert">
16 <i class="fas fa-exclamation-triangle me-2"></i>Esta acción realizará una "eliminación suave", el registro no se borrará permanentemente
17 </div>
18 <form method="post">
19 <div class="form-text">{{ csrf_token }}
20 </div>
21 <button type="submit" class="btn btn-danger me-2">
22 <i class="fas fa-trash-alt me-2"></i>Confirmar Eliminación
23 </button>

```

```

1  {% load crispy_forms_field %}
2
3  <div id="div_{{ field.id_for_label }}" class="form-group mb-3 {% if field.errors %}invalid{% endif %}">
4    <label for="{{ field.id_for_label }}" class="form-label {% if field.field.required %}required{% endif %}">
5      {{ field.label }}
6      {% if field.field.required %}<span class="asteriskField">*</span>{% endif %}
7    </label>
8    <div class="input-group">
9      <input type="{{ field.widget_type }}" name="{{ field.html_name }}" id="{{ field.id_for_label }}"
10         class="form-control {% if field.errors %}is-invalid{% endif %}"
11         {% if field.value %}value="{{ field.value }}" {% endif %}
12         {% if field.field.required %}required{% endif %}
13         placeholder="{{ field.field.widget.attrs.placeholder|default_if_none:'' }}"
14         {% if field.field.max_length %}maxlength="{{ field.field.max_length }}" {% endif %}
15         aria-describedby="{{ field.id_for_label }}-helptext">
16      <button type="button" class="btn btn-outline-secondary toggle-password" data-target="{{ field.id_for_label }}">
17        <i class="fas fa-eye"></i>
18      </button>
19      {% if field.errors %}
20        <div class="invalid-feedback">
21          {% for error in field.errors %}
22            {{ error }}
23          {% endfor %}
24        </div>
25      {% endif %}

```

Este es un template personalizado para el campo de contraseña, para incluir el ícono de "mostrar/ocultar".

ACTUALIZAMOS NUESTRO REPOSITORIOS

```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empaleados_dj521) λ git add .

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empaleados_dj521) λ git commit -m "APP Empleados - MTV"
[soft-control 8566f0d] APP Empleados - MTV
15 files changed, 866 insertions(+), 13 deletions(-)
create mode 100644 empleados/forms.py
create mode 100644 empleados/pydantic_models.py
create mode 100644 empleados/templates/base.html
create mode 100644 empleados/templates/empleado_confirm_delete.html
create mode 100644 empleados/templates/empleado_form.html
create mode 100644 empleados/templates/empleado_list.html
create mode 100644 empleados/templates/password_input.html
create mode 100644 empleados/urls.py
create mode 100644 static/css/style.css
create mode 100644 static/js/script.js

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empaleados_dj521) λ git push -u origin soft-control
Enumerating objects: 30, done.
Counting objects: 100% (30/30), done.
Delta compression using up to 8 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (23/23), 13.85 KiB | 1.26 MiB/s, done.
Total 23 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/efasoft/empleados.git
 0b4a9d1..8566f0d soft-control -> soft-control
branch 'soft-control' set up to track 'origin/soft-control'.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empaleados_dj521) λ

```

efasoft / empleados

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

empleados Public

Pin Unwatch 1 Fork 0 Star 0

soft-control

Go to file + Code

efasoft APP Empleados - MTV	8566f0d · 3 minutes ago
empleados	APP Empleados - MTV 3 minutes ago
formempleados	APP Empleados - MTV 3 minutes ago
static	APP Empleados - MTV 3 minutes ago
usuarios	Configuración Inicial del Proyecto ... 5 days ago
.gitignore	Formulario de Empleados con Dja... last week
Pydantic - Empleados.pdf	Configuración Inicial del Proyecto ... 5 days ago
manage.py	Configuración Inicial del Proyecto ... 5 days ago
requirements.txt	Configuración Inicial del Proyecto ... 5 days ago

About

Formulario de Empleados usando Pydantic con Django

Activity

0 stars

1 watching

0 forks

Releases

No releases published

[Create a new release](#)

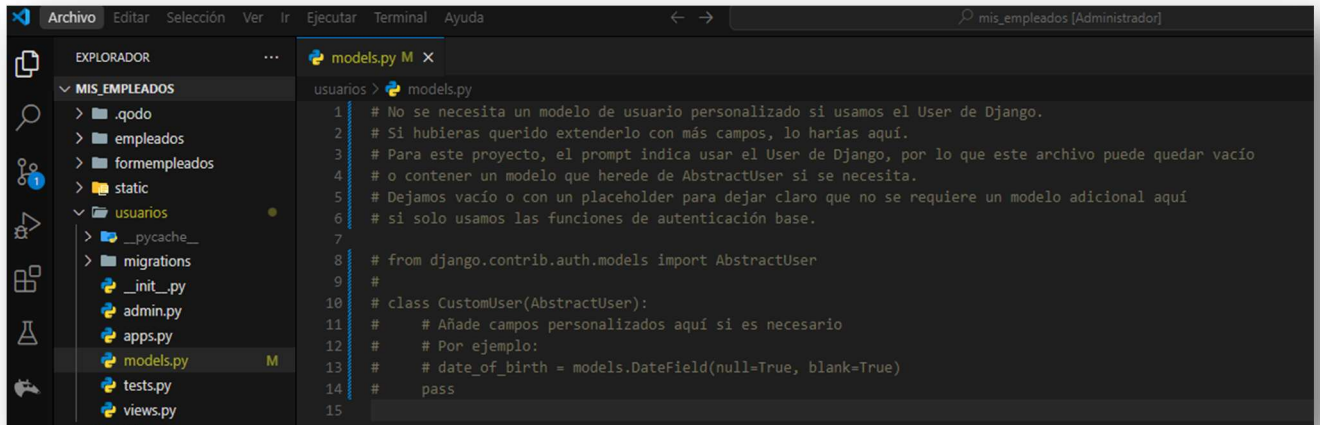
Packages

No packages published

[Publish your first package](#)

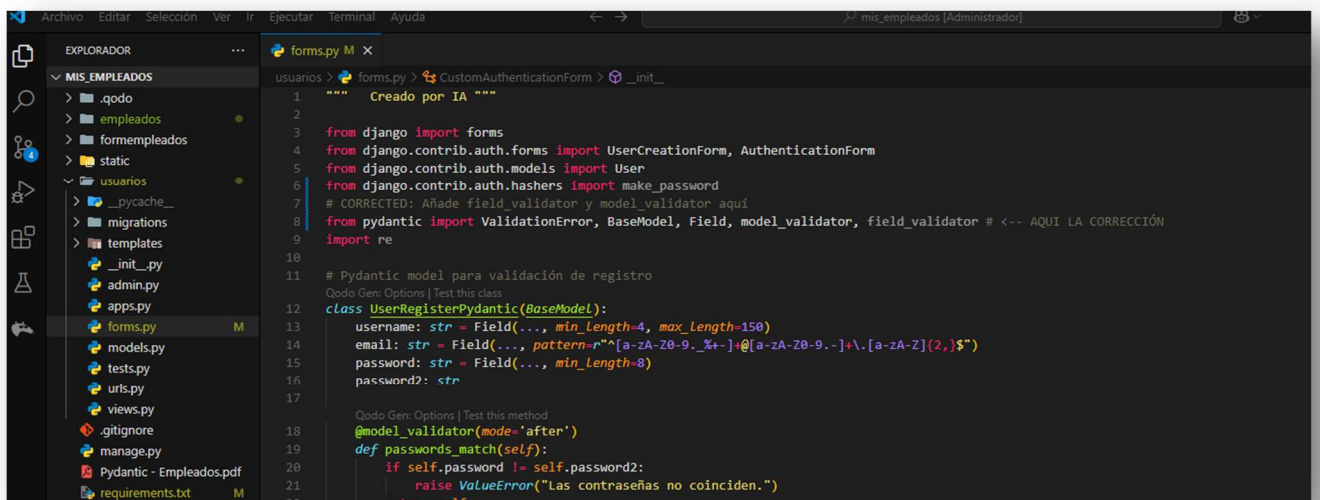
APP USUARIOS

ESTRUCTURA DE DATOS `models.py` IA



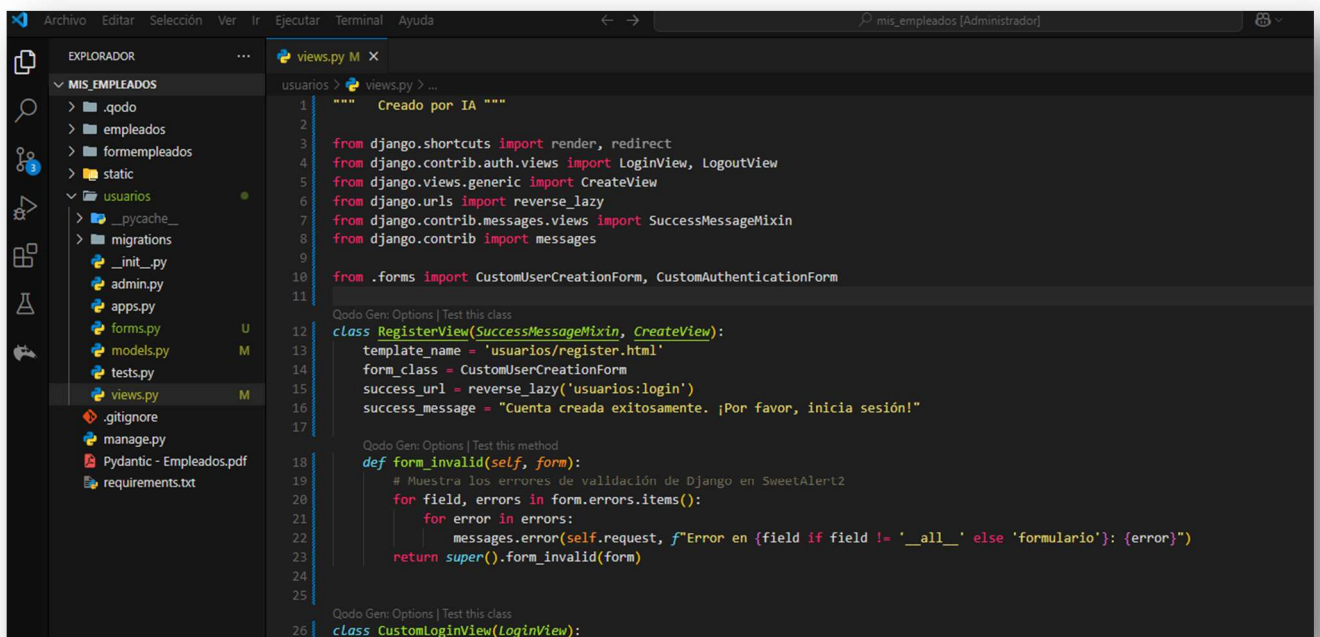
```
1 # No se necesita un modelo de usuario personalizado si usamos el User de Django.
2 # Si hubieras querido extenderlo con más campos, lo harías aquí.
3 # Para este proyecto, el prompt indica usar el User de Django, por lo que este archivo puede quedar vacío
4 # o contener un modelo que herede de AbstractUser si se necesita.
5 # Dejamos vacío o con un placeholder para dejar claro que no se requiere un modelo adicional aquí
6 # si solo usamos las funciones de autenticación base.
7
8 # from django.contrib.auth.models import AbstractUser
9 #
10 # class CustomUser(AbstractUser):
11 #     # Añade campos personalizados aquí si es necesario
12 #     # Por ejemplo:
13 #     # date_of_birth = models.DateField(null=True, blank=True)
14 #     pass
15
```

FORMULARIO `forms.py` IA

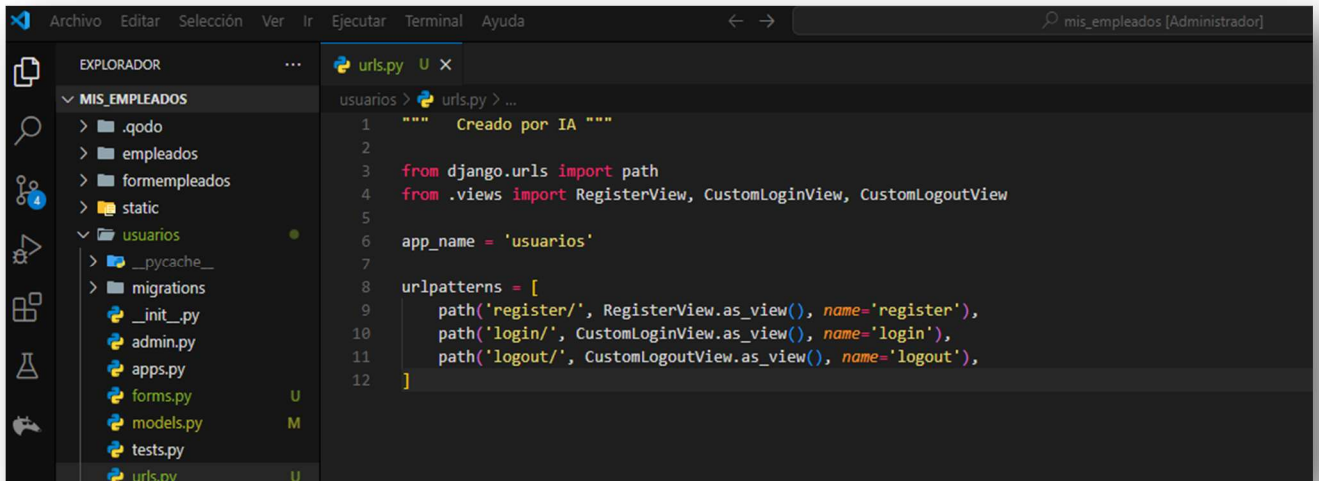


```
1 """ Creado por IA """
2
3 from django import forms
4 from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
5 from django.contrib.auth.models import User
6 from django.contrib.auth.hashers import make_password
7 # CORRECTED: Añade field_validator y model_validator aquí
8 from pydantic import ValidationError, BaseModel, Field, model_validator, field_validator # <-- AQUÍ LA CORRECCIÓN
9 import re
10
11 # Pydantic model para validación de registro
12 Qodo Gen: Options | Test this class
13 class UserRegisterPydantic(BaseModel):
14     username: str = Field(..., min_length=4, max_length=150)
15     email: str = Field(..., pattern=r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$")
16     password: str = Field(..., min_length=8)
17     password2: str
18
19 Qodo Gen: Options | Test this method
20 @model_validator(mode="after")
21 def passwords_match(self):
22     if self.password != self.password2:
23         raise ValueError("Las contraseñas no coinciden.")
24     return self
25
```

VISTA `views.py` IA



```
1 """ Creado por IA """
2
3 from django.shortcuts import render, redirect
4 from django.contrib.auth.views import LoginView, LogoutView
5 from django.views.generic import CreateView
6 from django.urls import reverse_lazy
7 from django.contrib.messages.views import SuccessMessageMixin
8 from django.contrib import messages
9
10 from .forms import CustomUserCreationForm, CustomAuthenticationForm
11
12 Qodo Gen: Options | Test this class
13 class RegisterView(SuccessMessageMixin, CreateView):
14     template_name = "usuarios/register.html"
15     form_class = CustomUserCreationForm
16     success_url = reverse_lazy('usuarios:login')
17     success_message = "Cuenta creada exitosamente. ¡Por favor, inicia sesión!"
18
19 Qodo Gen: Options | Test this method
20 def form_invalid(self, form):
21     # Muestra los errores de validación de Django en SweetAlert2
22     for field, errors in form.errors.items():
23         for error in errors:
24             messages.error(self.request, f"Error en {field if field != '__all__' else 'formulario'}: {error}")
25     return super().form_invalid(form)
26
27 Qodo Gen: Options | Test this class
28 class CustomLoginView(LoginView):
29
```

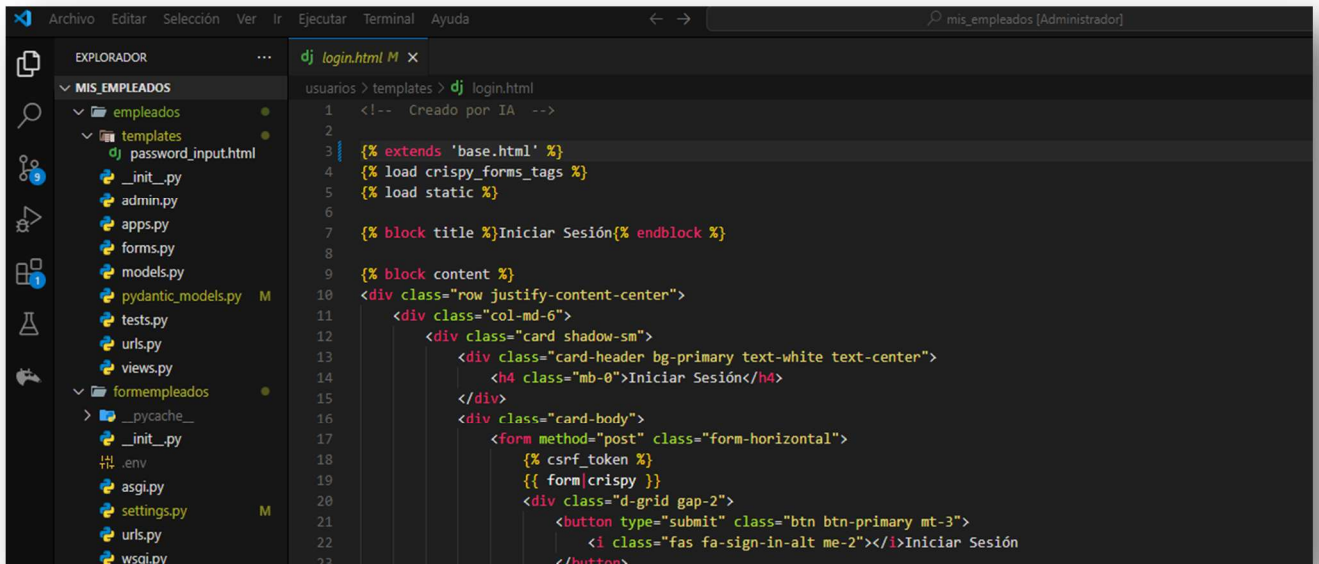



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure. The file `urls.py` is selected under the `usuarios` app. The main editor window shows the following Python code:

```

1  """ Creado por IA """
2
3  from django.urls import path
4  from .views import RegisterView, CustomLoginView, CustomLogoutView
5
6  app_name = 'usuarios'
7
8  urlpatterns = [
9      path('register/', RegisterView.as_view(), name='register'),
10     path('login/', CustomLoginView.as_view(), name='login'),
11     path('logout/', CustomLogoutView.as_view(), name='logout'),
12 ]
    
```

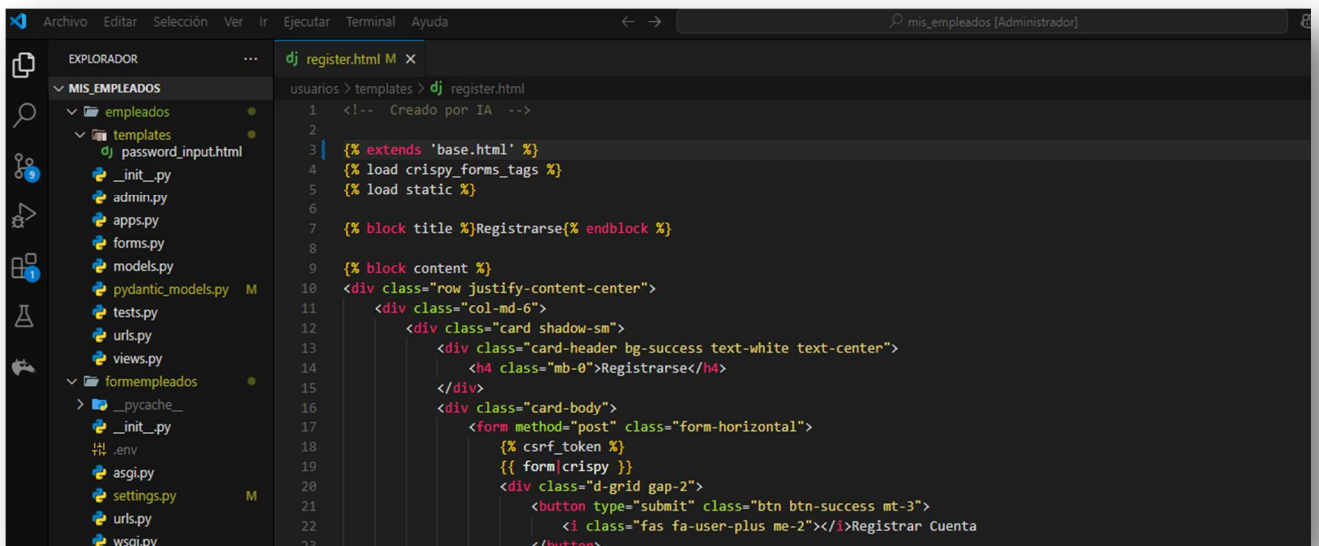
TEMPLATES



The screenshot shows the Visual Studio Code editor with the file explorer on the left. The file `login.html` is selected under the `templates` directory of the `usuarios` app. The main editor window shows the following Django template code:

```

1  <!-- Creado por IA -->
2
3  {% extends 'base.html' %}
4  {% load crispy_forms_tags %}
5  {% load static %}
6
7  {% block title %}Iniciar Sesión{% endblock %}
8
9  {% block content %}
10     <div class="row justify-content-center">
11         <div class="col-md-6">
12             <div class="card shadow-sm">
13                 <div class="card-header bg-primary text-white text-center">
14                     <h4 class="mb-0">Iniciar Sesión</h4>
15                 </div>
16                 <div class="card-body">
17                     <form method="post" class="form-horizontal">
18                         {% csrf_token %}
19                         {{ form|crispy }}
20                     <div class="d-grid gap-2">
21                         <button type="submit" class="btn btn-primary mt-3">
22                             <i class="fas fa-sign-in-alt me-2"></i>Iniciar Sesión
23                         </button>
    
```



The screenshot shows the Visual Studio Code editor with the file explorer on the left. The file `register.html` is selected under the `templates` directory of the `usuarios` app. The main editor window shows the following Django template code:

```

1  <!-- Creado por IA -->
2
3  {% extends 'base.html' %}
4  {% load crispy_forms_tags %}
5  {% load static %}
6
7  {% block title %}Registrarse{% endblock %}
8
9  {% block content %}
10     <div class="row justify-content-center">
11         <div class="col-md-6">
12             <div class="card shadow-sm">
13                 <div class="card-header bg-success text-white text-center">
14                     <h4 class="mb-0">Registrarse</h4>
15                 </div>
16                 <div class="card-body">
17                     <form method="post" class="form-horizontal">
18                         {% csrf_token %}
19                         {{ form|crispy }}
20                     <div class="d-grid gap-2">
21                         <button type="submit" class="btn btn-success mt-3">
22                             <i class="fas fa-user-plus me-2"></i>Registrar Cuenta
23                         </button>
    
```

ACTUALIZAMOS NUESTRO REPOSITORIOS

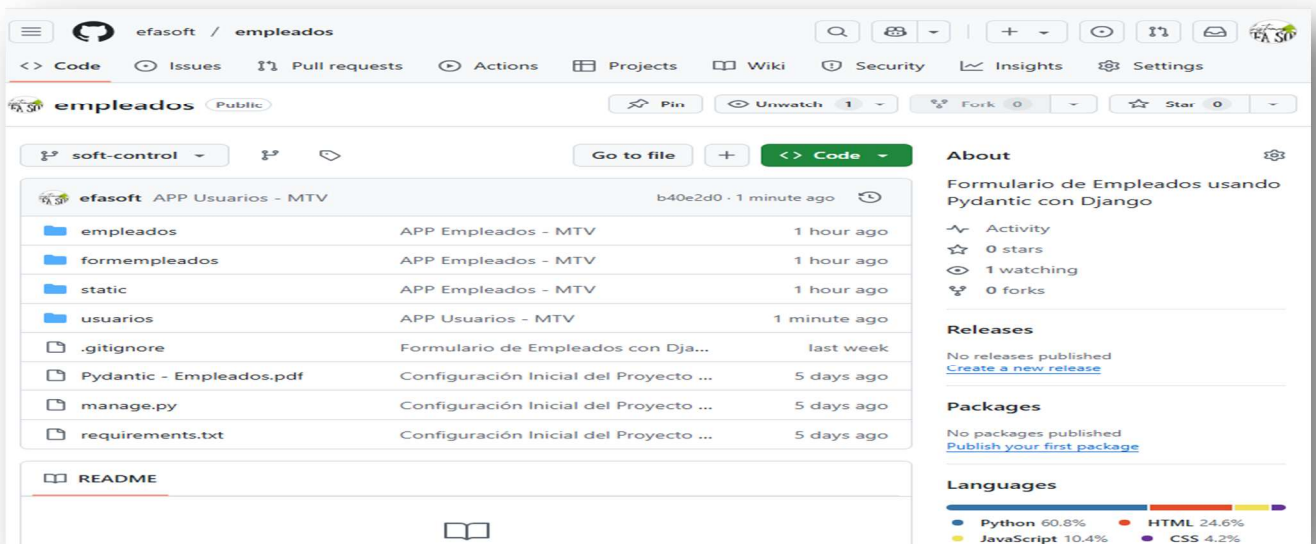
```
Cmder

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git add .

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git commit -m "APP Usuarios - MTV"
[soft-control b40e2d0] APP Usuarios - MTV
6 files changed, 213 insertions(+), 4 deletions(-)
create mode 100644 usuarios/forms.py
create mode 100644 usuarios/templates/login.html
create mode 100644 usuarios/templates/register.html
create mode 100644 usuarios/urls.py

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git push -u origin soft-control
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 3.68 KiB | 1.23 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/efasoft/empleados.git
 8566f0d..b40e2d0 soft-control -> soft-control
branch 'soft-control' set up to track 'origin/soft-control'.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```



MIGRACIONES MODELOS empleados y usuarios

Desde nuestro terminal ejecutamos:

```
py manage.py makemigrations
py manage.py migrate
```

```
Cmder

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py makemigrations
System check identified some issues:

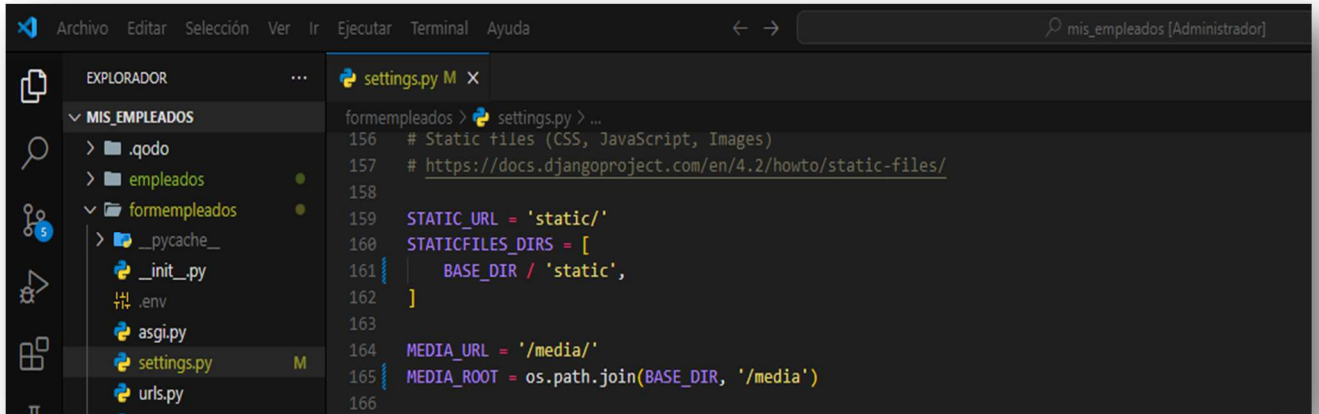
WARNINGS:
?: (staticfiles.W004) The directory 'D:\Desarrollo Py\mis_empleados\empleados\static' in the STATICFILES_DIRS setting does not exist.
Migrations for 'empleados':
  empleados\migrations\0001_initial.py
    - Create model Empleado

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py migrate
System check identified some issues:

WARNINGS:
?: (staticfiles.W004) The directory 'D:\Desarrollo Py\mis_empleados\empleados\static' in the STATICFILES_DIRS setting does not exist.
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, empleados, sessions
Running migrations:
  Applying empleados.0001_initial... OK

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

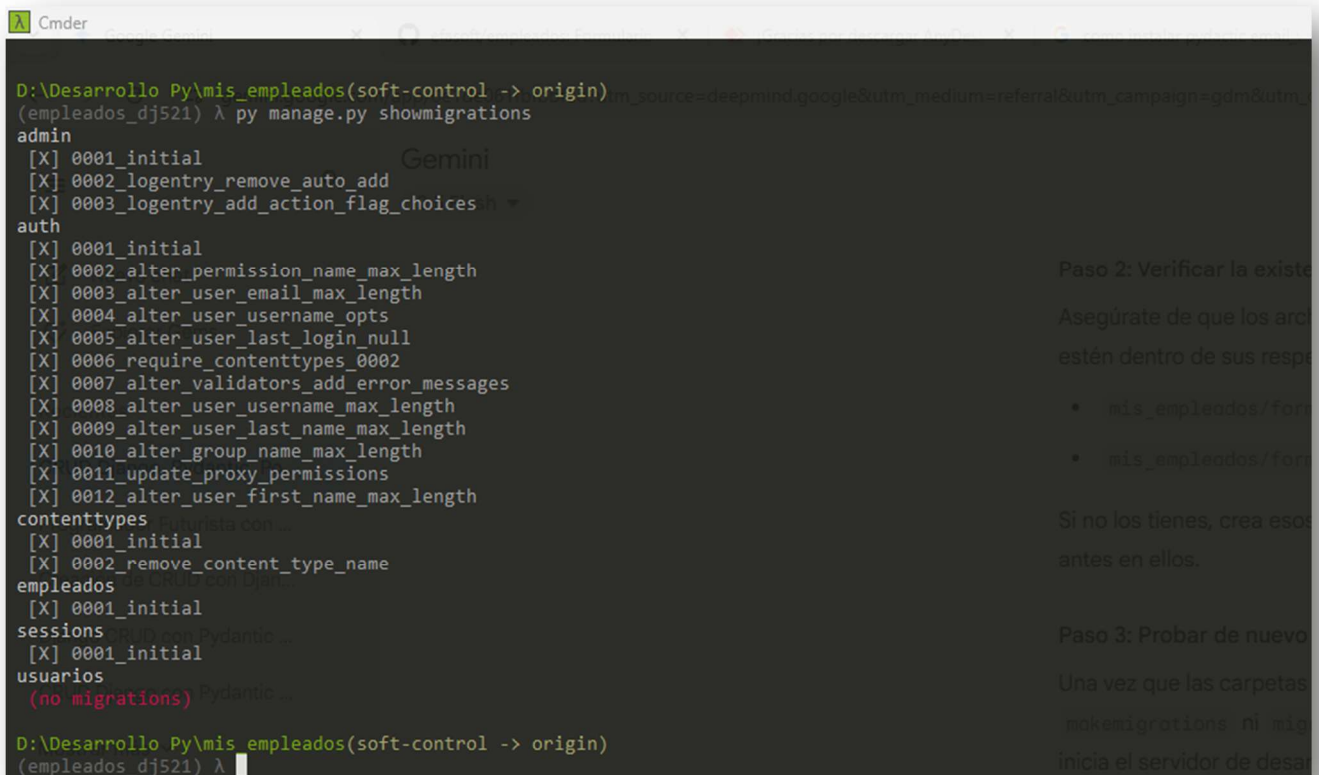
Corregimos el error : **Directorio no existe**



```
EXPLORADOR MIS_EMPLEADOS
  > .qodo
  > empleados
  > formempleados
    > __pycache__
    > __init__.py
    > .env
    > asgi.py
    > settings.py M
    > urls.py

settings.py M
156 # Static files (CSS, JavaScript, Images)
157 # https://docs.djangoproject.com/en/4.2/howto/static-files/
158
159 STATIC_URL = 'static/'
160 STATICFILES_DIRS = [
161     BASE_DIR / 'static',
162 ]
163
164 MEDIA_URL = '/media/'
165 MEDIA_ROOT = os.path.join(BASE_DIR, '/media')
166
```

Comprobamos la migración: **py manage.py showmigrations**



```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin) m_source=deepmind.google&utm_medium=referral&utm_campaign=gdm&utm_
(empleados_dj521) λ py manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
empleados
[X] 0001_initial
sessions
[X] 0001_initial
usuarios
(no migrations)

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

levantamos nuestro servidor de desarrollo : **py manage.py runserver**



```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin) m_source=deepmind.google&utm_medium=referral&utm_campaign
(empleados_dj521) λ py manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
May 28, 2025 - 16:57:52
Django version 4.2.21, using settings 'formempleados.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```


← → ↻ 127.0.0.1:8000/usuarios/login/?next=/empleados/ 🔍 ☆ 📁 E ⋮

Gestión de Empleados

Iniciar Sesión

Nombre de usuario*

Password*

➔ Iniciar Sesión

+ Registrarse

Tenemos en marcha nuestro proyecto.

ACTUALIZAMOS NUESTRO REPOSITORIOS

```

Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git add .

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git commit -m "Proyecto terminado... iniciando pruebas"
[soft-control a9791a7] Proyecto terminado... iniciando pruebas
11 files changed, 111 insertions(+), 61 deletions(-)
delete mode 100644 Pydantic - Empleados.pdf
create mode 100644 empleados/migrations/0001_initial.py
create mode 100644 static/img/rutas.jpg

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ git push -u origin soft-control
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 8 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (20/20), 48.38 KiB | 9.68 MiB/s, done.
Total 20 (delta 11), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (11/11), completed with 11 local objects.
To https://github.com/efasoft/empleados.git
 b40e2d0..a9791a7 soft-control -> soft-control
branch 'soft-control' set up to track 'origin/soft-control'.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
  
```

efasoft / empleados

🔍
📁
+
🔄
🔗
📧
👤

<> Code
🕒 Issues
🔗 Pull requests
🎬 Actions
📁 Projects
📖 Wiki
🔒 Security
📊 Insights
⚙️ Settings

📁 empleados
Public
📌 Pin
👁️ Unwatch 1
🍴 Fork 0
⭐ Star 0

👤 soft-control
🗨️

Go to file
+
<> Code

📁 efasoft Proyecto terminado... iniciando pruebas a9791a7 · 1 minute ago 🕒

📁 empleados	Proyecto terminado... iniciando pr...	1 minute ago
📁 formempleados	Proyecto terminado... iniciando pr...	1 minute ago
📁 static	Proyecto terminado... iniciando pr...	1 minute ago
📁 usuarios	Proyecto terminado... iniciando pr...	1 minute ago
📄 .gitignore	Formulario de Empleados con Dja...	last week
📄 manage.py	Configuración Inicial del Proyecto ...	5 days ago
📄 requirements.txt	Proyecto terminado... iniciando pr...	1 minute ago

📖 README

📖 Add a README

About

Formulario de Empleados usando Pydantic con Django

🔄 Activity

⭐ 0 stars

👁️ 1 watching

🍴 0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

Python 60.8% HTML 24.6% JavaScript 10.4% CSS 4.2%